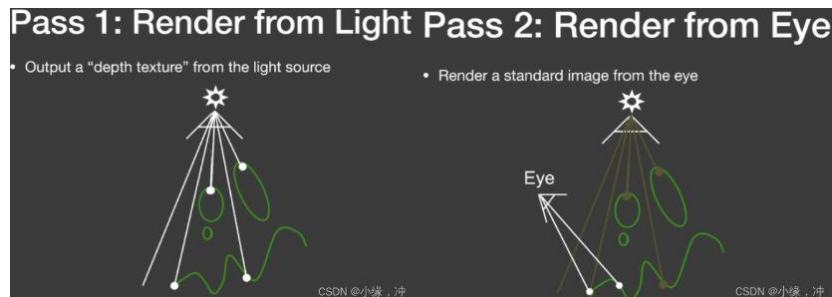
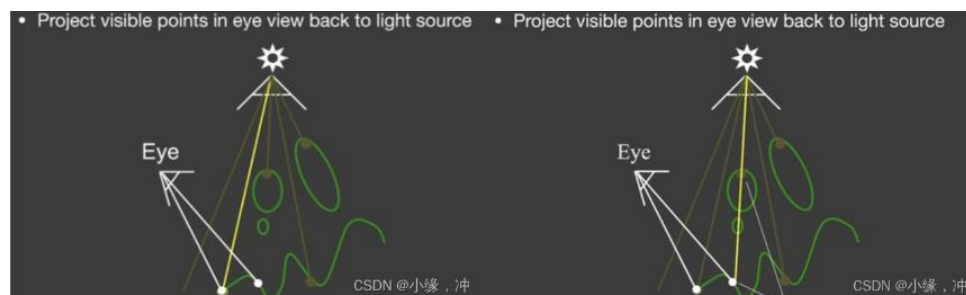


一、shadow（阴影）

a. shadow map 回顾:

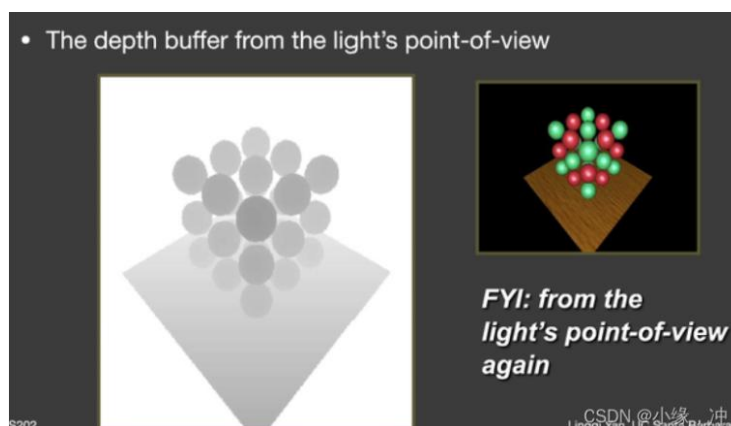


我们先从 Light 处看向场景并输出一个从 light 处看向场景所生成的深度图得到一张 texture, 也就是所谓的 shadow map. 而后从 camera 处看向场景渲染一遍, 并参考 1-pass 生成的 shadow map 去判断物体是否在阴影中.



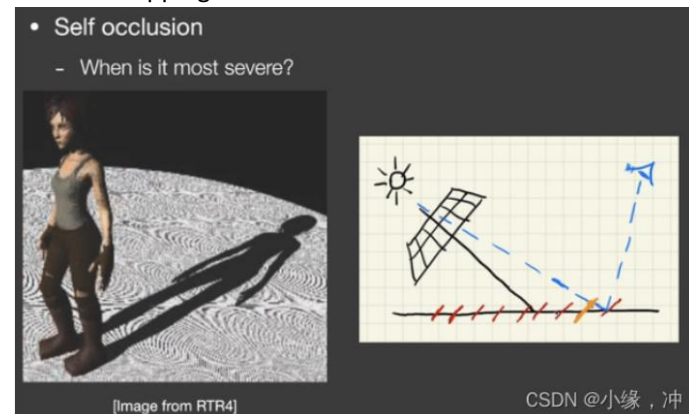
左图中, 点连向 light, 其在 shadow map 的最浅深度 等于 图中从点连到 light 处的深度, 所以点不在阴影中. 右图, 点连向 light, 其在 shadow map 的最浅深度 小于 图中从点连到 light 处的深度, 所以点在阴影中.

shadow mapping 是一个完全在图像空间中的算法, 其优点: 一旦 shadow Map 已经生成, 就可以利用 shadow map 来获取场景中的几何表示而不是场景中的几何. 其缺点: 会产生自遮挡和走样现象.



从 Light 处看向场景生成的是 shadow map, 并不是 Shading 的结果, shadowmap 颜色深的表示值比较小, 也就是离 light 近, 颜色浅的就是离 light 远, 值大.

Shadow Mapping 存在的问题:



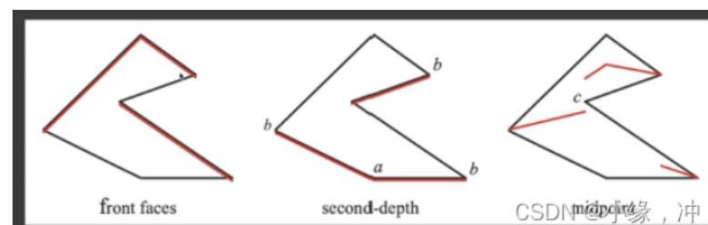
自遮挡的现象：主要是由于 shadow map 记录的深度的不连续造成的地板上的东西会感觉像摩尔纹,但并不是,它是由数值精度造成的一种现象。我们生成了一个 Shadow map,那么这个 Shadow map 肯定有自己的分辨率,从 Light 处看向场景时,沿某一像素看过去我们看到的位置就认为是像素所代表的深度,记录的深度值是离散的,不连续的,如图中红色斜线所示。图中眼睛看到的蓝色点,实际上不会被遮挡的,但其实际深度大于 shadow map 上对应像素记录的深度,造成了被遮挡的假象,由此产生了自遮挡现象。在 light 与平面趋于平行时候最严重。

消除自遮挡现象:

- 1 使用更高分辨率的 shadow map, 可以从一定程度上缓解自遮挡现象, 因为高分辨率会使记录的深度变化更加平滑。
- 2 在深度比较时添加 bias, 在深度比较时不计算图中黄色部分, 就可以解决自遮挡问题。当 Light 处垂直于场景时,我们可以让这个遮挡区域尽可能小一点,当 light 趋近于平行场景时,我们让这个区域尽可能大一点。
- 3 在生成 shadow map 的时候, 不仅记录最小深度, 还需要记录第二小的深度。



如果 bias 太大, 也会出现另一个非常严重的问题——物体与阴影的割裂。



在生成 shadow map 的时候, 不仅记录最小深度, 还需要记录第二小的深度, 在深度比较时, 使用前面二者的平均值, 可以有效地消除自遮挡现象。此方法不使用 bias。但是开销大

走样-锯齿: shadow map 本身就存在分辨率, 当分辨率不够大自然会看到锯齿:



b. Shadow mapping math

在实时渲染中,我们只关心近似约等,我们不考虑不等的情况,因此我们将某些不等式当约等式来使用.如:

$$\int_{\Omega} f(x)g(x) dx \approx \frac{\int_{\Omega} f(x) dx}{\int_{\Omega} dx} \cdot \int_{\Omega} g(x) dx$$

满足下列两个条件之一时, 实时渲染领域认为他们近似相等:

1. $g(x)$ 的积分域很小
2. $f(x)$ 或者 $g(x)$ 足够 smooth, 变化不大

以此方法对渲染方程进行处理

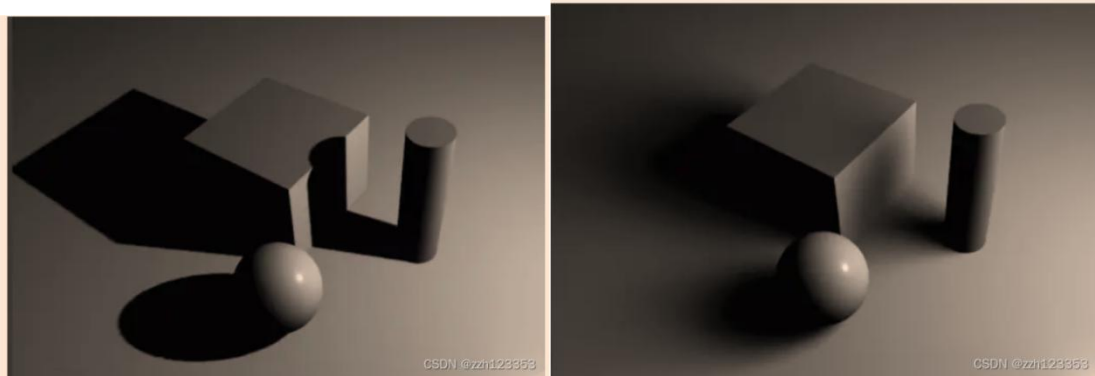
- Recall: the rendering equation with explicit visibility
$$L_o(p, \omega_o) = \int_{\Omega^+} L_i(p, \omega_i) f_r(p, \omega_i, \omega_o) \cos \theta_i V(p, \omega_i) d\omega_i$$
- Approximated as
$$L_o(p, \omega_o) \approx \frac{\int_{\Omega^+} V(p, \omega_i) d\omega_i}{\int_{\Omega^+} d\omega_i} \cdot \int_{\Omega^+} L_i(p, \omega_i) f_r(p, \omega_i, \omega_o) \cos \theta_i d\omega_i$$

积分中多了一项 visibility, 把 visibility 看作是 $f(x)$, 提取出来并作归一化处理。红色区域部分为 visibility (能见度), 那么剩下的 $g(x)$ 部分, 也就是 shading 的结果。意义就是, 我们计算每个点的 shading, 然后去乘这个点的 visibility 得到的就是最后的渲染结果。也就是 shadow mapping 的基本思想。

那么什么时候这个约等式比较正确呢?

- a) 我们要控制积分域足够小, 也就是说我们只有一个点光源或者方向光源。
- b) 我们要保证 shading 部分足够光滑, 也就是说 brdf 的部分变化足够小, 那么这个 brdf 部分是 diffuse 的。
- c) 我们还要保证光源各处的 radiance 变化也不大, 类似于一个面光源。

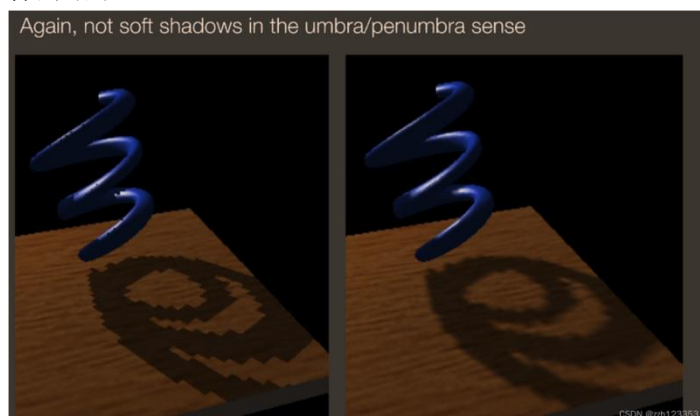
c. PCF-百分比渐进滤波



软阴影的效果更加真实，为了实现软阴影效果，我们引入了 PCF-Percentage closer filtering
PCF 初衷是为了抗锯齿，后来发现可以用在软阴影中，通过把 shadow 结果做一个加权平均 (filtering)。

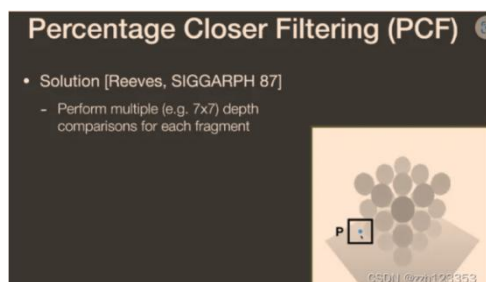
在哪 Filter？：

1.不是在最后生成的阴影结果中 filter，而是在做阴影判断的时候进行 filter。在原本的锯齿的结果中进行 filter，跟 Games101 反走样一样，在一个走样的结果中做模糊，还是会得到走样的结果。



2.也不是 Filter Shadow Map:如果直接 Filter Shadow Map，那么边界会变得模糊起来，而且在第二次 pass 的时候得到的结果仍然是非 0 即 1，最后得到的还是硬阴影。

我们之前在做点是否在阴影中时,把 shading point 连向 light 然后跟 Shadow map 对应的这一点深度比较判断是否在阴影内,之前我们是做一次比较,这里的区别是,对于这个 shading point 我们仍要判断是否在阴影内,但是我们把其投影到 light 之后不再只找其对应的单个像素,而是找其周围一圈的像素,把周围像素深度比较的结果加起来平均一下，就得到一个 0-1 之间的数，就得到了一个模糊的结果。



如图,蓝点是本来应该找的单像素,现在我们对其周围 3×3 个像素的范围进行比较,由于是在 Shadow map 上,因此每个像素都代表一个深度,我们让在 shadow map 上范围内的每个像素都与 shading point 的实际深度进行一下比较,如果 shadow map 上范围内的像素深度小于 shading point 的实际深度,则输出 1,否则输出 0.



d. PCSS (Percentage Closer Soft Shadows)

我们会发现如果 filter size 越大, 阴影本身越软, 所以这个方法也就可以去绘制软阴影, 也就是 pcss 技术。



在这个途中, 可以看到笔尖的阴影十分锐利(硬阴影), 因此我们认为阴影接受物与阴影投射物之间的距离越小, 阴影就越硬

因此,我们要解决的一个问题是我们如何决定一个软阴影的半影区。换句话说,就是 filter size 有多大的问题:

- Key conclusion
 - Filter size $\leftrightarrow$ blocker distance
 - More accurately, **relative average** projected blocker depth!
- A mathematical "translation"

$$w_{Penumbra} = (d_{Receiver} - d_{Blocker}) \cdot w_{Light} / d_{Blocker}$$

根据相似三角形即可得到对应的相似关系, filter size 取决于 d-Blocker, 接下来就是如何确定 blocker 的距离呢?

不能直接使用 shadow map 中对应单个点的深度来代表 blocker 距离,因为如果该点的深度与周围点的深度差距较大(遮挡物的表面陡峭或者对应点正好有一个孔洞),将会产生一个错误的效果,我们选择使用平均遮挡距离来代替,所以平常我们指的 blocker depth 其实是 Average blocker depth.

blocker 上的每个点距离光源的距离是不同的,深度也是不一样的。这里我们采用取平均深度的方式来表示 blocker 的深度。

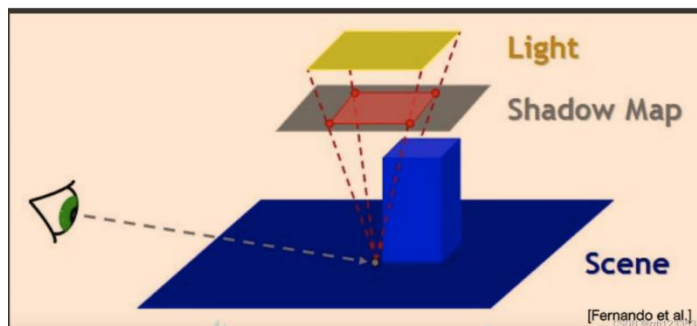
求 blocker 距离的方法如下:

首先,我们把目标 shading point 转换到 light space 找到 shading point 在 shadow map 上对应的像素。

如果 shading point 的深度大于这个 shadow map 上点对应的深度,则说明 shadow map 上的点就是一个 Blocker,然后我们取 shadow map 上这个点(像素)周围的一些像素,找出能够挡住 shading point 的点的像素,并求出他们的深度平均值作为 blocker 的深度。

第一种,就是自己规定一个,比如 $4 * 4$, $16 * 16$,比较简单但不实用。

第二种,是通过计算得到一个范围大小:



离光源越近,遮挡物会少,所以需要在 Shadow map 上的一个大区域内查找 blocker.

这样我们就得到了 PCSS 的三个步骤:

- 1.寻找 blocker,并计算平均深度。
- 2.通过 blocker 深度计算 filter size。
- 3.按照 PCF 方式绘制软阴影。

PCSS 本质上就是求出了阴影中需要做 PCF 的半影部分后再进行 PCF 的计算,这样动态调节了半影范围,也就是动态设置了 PCF 的搜索范围,这样我们的硬阴影部分清晰,软阴影部分模糊,动态的实现不错的软阴影效果。

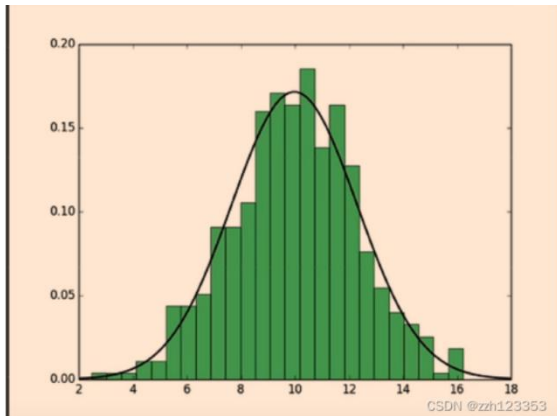
可以看到,1、3 都需要做一个范围查询,计算平均值,这样会使得计算速度严重下降,如何解决呢?就有工业界大佬提出了 VSSM 的解决方法

e. VSSM(Variance soft shadow mapping)

我们回到第三步 PCF 之中,我们要在 shadow map 上对其周围的一圈像素的各个最小深度与 Shading point 比较,从而判断是否遮挡,也就是要求出范围内有百分之多少的像素比它浅。

这个过程很像在考试成绩出来后,你知道了自己的成绩,你想知道自己在班级中的排名,因此你需要知道班级中所有人的成绩从而进行比较来判断自己是百分之几,这就是 PCF 的做法。

但现在我们就是为了避免这种时间消耗大的做法.那么一个不错的办法就是,对班级所有人的成绩做成一个直方图,根据直方图我们可判断出自己的成绩排名。



如果我们不需要那么准的话就可以当做一个正态分布,正态分布就只需要方差和平均值就能得出,更加的方便快捷,这也就是 **VSSM** 的核心思想,通过正态分布来知道自己大约占百分之几.

VSSM 的 **key idea** 是快速计算出某一区域内的均值和方差.

求均值: 对于快速的求一个范围内的求均值,我们可以想到在 **games101** 中学到的 **mipmap** 方法.但是 **mipmap** 毕竟是不准的,而且只能在正方形区域内查询.因此引入 **Summed Area Tables (SAT)**

求方差: **VSSM** 用结合了期望与方差之间的关系的一个公式来得到方差:

- Variance

- $\text{Var}(X) = E(X^2) - E^2(X)$

- So you just need the mean of (depth^2)

CSDN @zzh123353

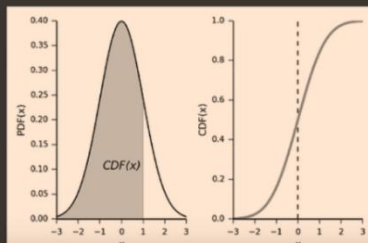
此为止我们就快速获得了均值和方差。那么回到问题本身:

有多少百分比的像素是比 **Shading point** 大,也就是不会挡住 **Shading point** 的只需要计算出下图 **PDF** 中白色面积 的值就行了。

有多少百分比的像素是比 **Shading point** 小,也就是会挡住 **Shading point** 的只需要计算出下图 **PDF** 中灰色面积 的值就行了。

- Back to the question

- Percentage of texels that are closer than the shading point
 - You want to calculate the shade's area
 - Accurate answer exists (hint: What's the CDF of a Gaussian PDF?)



CSDN @zzh123353

PDF: 概率密度函数 (**probability density function**), 在数学中, 连续型随机变量的概率密度函数是一个描述这个随机变量的输出值, 在某个确定的取值点附近的可能性的函数。

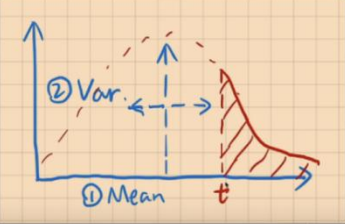
CDF: 累积分布函数 (**cumulative distribution function**), 又叫分布函数, 是概率密度函数的积分, 能完整描述一个实随机变量 **X** 的概率分布。

其实想知道 CDF，也就是求出 PDF 曲线下对应的面积。

对于一个通用的高斯的 PDF，对于这类 PDF，可以直接把 CDF 结果，输出为一个表，叫误差函数 Error Function，误差函数有数值解，但是没有解析解，在 C++ 中的函数 `ERF(lower_limit,upper_limit)` 函数可以计算 CDF。

- It doesn't have to be too accurate!
 - Chebyshev's inequality (one-tailed version, for $t > \mu$)
$$P(x > t) \leq \frac{\sigma^2}{\sigma^2 + (t - \mu)^2}$$
$$\mu : \text{mean}$$
$$\sigma^2 : \text{variance}$$

Doesn't even assume Gaussian distribution!



我们用切比雪夫不等式,来近似地求出在知道 期望 和 方差 时候,不考虑是不是正态分布,图中红色面积不会超过等式右边,这里使用了一个 Trick 的方法,将这个不等式近似为相等.那么就可以通过均值和方差获得图中红色面积的值。因此就可以不用计算 CDF 了,而是通过 $1 -$ 求出的 $x > t$ 的面积值得到 CDF.但是切比雪夫不等式有一个苛刻的条件: t 必须在均值右边,也就是 t 大于均值。

加速第三步总结:

- 我们通过生成 shadow map 和 square-depth map 得到期望值的平方和平方值的期望再根据公式得到方差
- 通过 mipmap 或者 SAT 得到期望
- 得到期望和方差之后,根据切比雪夫不等式近似得到一个 depth 大于 shading point 点深度的面积.,也就是求出了未遮挡 Shading point 的概率,从而可以求出一个在 1-0 之间的 visibility.也就是省去了在这个范围内进行采样或者循环的操作,大大加速了第三步.

如果场景/光源出现移动 就需要更新 MIPMAP, 本身还是有一定的开销, 但是生成 MIPMAP 硬件 GPU 支持的非常到位, 生成非常快(几乎不花时间), 而是 SAT 会慢一点, 这个后面进行分析。

到目前为止 VSSM 也只解决了第三步 PCF Filter 的问题, PCSS 在第一步要要求在范围内将所有像素的深度走一遍从而求平均遮挡深度 Average Blocker Depth 的问题并未解决。

- Key idea
 - Blocker ($z < t$), avg. z_{occ} (we want to compute)
 - Non-blocker ($z > t$), avg. z_{unocc}

$$\frac{N_1}{N} z_{unocc} + \frac{N_2}{N} z_{occ} = z_{Avg}$$

4	4	8	8	8
4	4	8	8	8
4	4	6	8	8
6	6	6	8	9
8	8	8	9	9

CSDN @zzh123353

要计算的是平均深度实际就是蓝色区域平均值, 即遮挡物的平均深度

非遮挡像素占的比例 * 非遮挡物的平均深度 + 遮挡像素占的比例 * 遮挡物的平均深度 = 总区域内的平均深度。

总区域内的平均深度我们用 mipmap 或者 SAT 去求,然后用 shadow map 和 square-depth map 方差,最后根据切比雪夫不等式近似求出非遮挡像素占的比例和遮挡像素占的比例

$$\frac{N1}{N} = P(x > t)$$

$$\frac{N2}{N} = 1 - P(x > t)$$

为了解出 Zocc 我们做一个大胆的假设,我们认为非遮挡物的平均深度即 $Z_{unocc} = \text{shading point}$ 的深度,至此就可以解出方程求出平均深度.但是接受平面是曲面或者与光源不平行的时候就会出问题。

VSSM 的做法实在是十分聪明,采用了非常多的大胆假设,同时非常的快,没有任何噪声,本质上其实也没有用正态分布,是直接切比雪夫不等式来进行近似。但是现在最主流的方法仍然是 PCSS (噪声版本,即取部分样本来代替整体),因为人们对噪声的容忍度变高加上降噪的技术越来越高明,因此大多数人采用 PCSS。

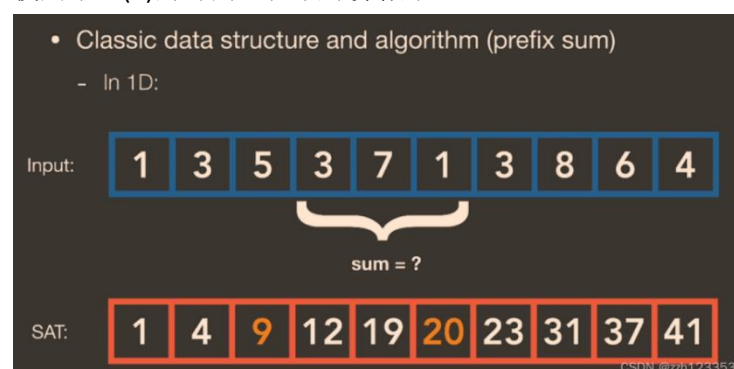
f. SAT

MIPMAP,我们在 GAMES101 里学过,他是一个快速的,近似的,正方形的范围查询,由于他要做插值,因此即便是方形有时也会不准确.同时当插值的范围不是 2 的次方时,也就是在两个 MIPMAP 之间时,还要再进行一次插值,也就是“三线性插值”,这样会让结果更加不准确,因此局限性太大且准确度也不算高。

但是 SAT 是百分百准确的一个数据结构.SAT 的出现是为了解决范围查询(在区域内快速得到平均值),并且,范围内求平均值是等价于范围内求和的,毕竟总和除以个数=平均值。

在 1 维情况下其实就是一维数组,SAT 这种数据结构就是做了预处理,也就是在得到一维数组时,先花费 $O(n)$ 的时间从左到右走一遍,并且在走的同时把对应的累加和存入数组 SAT 中,那么 SAT 上任意的一个元素就等于原来数组从最左边的元素加到这个元素的和,如图 SAT 数组第 2 个元素表示原数字前 2 个元素之和。

那么查询下图中 SUM 区域总和时候,就是 SAT 数组中第六个元素减去第三个元素。相当于使用了 $O(n)$ 的时间,把预计算做了一遍。



那么在 2 维 (二维数组) 情况下 有一个任意矩形 (横平竖直), 蓝色矩形内的总和为两个绿色矩形内的总和减去两个橙色矩形内的总和。同样可以采用 SAT 的方法, 做一个表, 做出从左上角到这个元素的和。因此只需要查表 4 次就可以得出精准的区域求和。

在 2 维情况下, 可以通过建立 m 行中每行的 SAT 和在每行的 sat 基础上再建立 n 列中每列的 SAT, 最终可以获得一个 2 维的 SAT 因此最终的 SAT, 但是由于 gpu 的并行度很高, 行与行或列与列之间的 sat 可并行, 因此具有 $m \times n$ 的时间复杂度。

- Classic data structure and algorithm
 - In 2D:

[Gamboa et al.]

- Note: accurate, but need $O(n)$ time and storage to build
 - Storage might not be an issue
 - Can we speed up building SAT?

CSDN @zzh123353

g. Monment Shadow Mapping

VSSM 是为了解决 PCSS 的问题,但 vssm 由于做了很多假设,当假设不对的时候会有问题。

- Is a normal distribution always good enough to approximate the distribution of fragments' distances?

CSDN @zzh123353

比如右图,只有三个片的遮挡的情况下,那么深度的分布就在这三个遮挡度深度周围,形成了三个峰值,自然就会出现假设描述的不准。

- Issues if the depth distribution is inaccurate
 - Overly dark: may be acceptable
 - Overly bright: LIGHT LEAKING!

CSDN @zzh123353

不是正态分布强行按正态分布算就会出现漏光和过暗的结果。在阴影的承接面不是平面的情况下也会出现阴影断掉的现象。

Revisit: VSSM

- Limitations?
 - Light leaking
 - non-planarity artifact
- Chebychev is to blame?
 - Only valid when $t > Z_{avg}$
- Can we do better?

[https://developer.nvidia.com/gpugems3/gpugems3_ch08.html]

[Yang et al.]

我们看图中小车可以发现,在车的底部阴影部分出现了一部分偏白的阴影,这是因为车是一个镂空的状态,如果从底部向 LIGHT 处看去会发现底板遮挡一部分,车顶附近会遮挡一部分,这就导致了不是正态分布情况,因此才出现 light leaking 这种情况.

因此人们为了避免 VSSM 中不是正态分布情况下的问题,就引入了更高阶的 moments 来得到更加准确的深度分布情况.想要描述的更准确,就要使用更高阶的 moment(矩),矩的定义有很多,最简单的矩就是记录一个数的次方, VSSM 就等于用了前两阶的矩。这样多记录几阶矩就能得到更准确的结果。

Moment Shadow Mapping

- Moments
 - Quite a few variations on the definition
 - We use the simplest: $x, x^2, x^3, x^4, \dots$
 - So, VSSM is essentially using the first two orders of moments

CSDN @zzh123353

如果保留前 M 阶的矩,就能描述一个阶跃函数,阶数等 $M/2$,就等于某种展开。越多的阶数就和原本的分布越拟合。一般来说 4 阶就够用。

Moment Shadow Mapping

- What can moments do?
 - Conclusion: first m orders of moments can represent a function with $m/2$ steps
 - Usually, 4 is good enough to approximate the actual CDF of depth dist.
 - How to restore a CDF whose moments match the given moments

CSDN @zzh123353

h. SDF

距离场, SDF 大多用在与几何之类的方向上,比如我有两个物体的 SDF,我们将两个 SDF 进行 Blend 操作,从而得到一个新的 SDF,也就得到了一个新的几何物体,因为当距离 0 为实,即可认为是物体的边界, SDF 可以很准确地反应物体的边界。几何转换 SDF 的时候,可以很好地把几何进行过渡。

From GAMES101: Distance Functions

- Can blend any two distance functions d_1, d_2

i. Ray Marching

假设我们已经知道场景的 SDF,现在有一根光线,我们试图让光线和 SDF 所表示的隐含表面进行求交,也就是我们要用 sphere tarcing(ray marching)进行求交.

RayMarching 可以理解为一个贪心思想:

任意一点的 SDF 我们是已知的,假如在 P 点,此时我们以 P 为中心点,以他

的 SDF 值为半径画圆、球，考虑 2D 的情况，在圆内部，不可能与物体相交，这是一个安全距离，那么每次都利用上这个安全距离，直到与物体相交。

- Usage 1
 - Ray marching (sphere tracing) to perform ray-SDF intersection
 - Very smart idea behind this:
 - The value of SDF == a "safe" distance around
 - Therefore, each time at p, just travel SDF(p) distance



如果在超一方向 trace 非常远的距离但仍然什么都没 trace 到,此时就可以舍弃这条光线,也就是停止了.

j. Generate Soft Window

将安全距离延申，可以得到 safe angle，安全角的概念

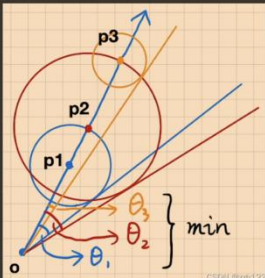
- Usage 2
 - Use SDF to determine the (approx.) percentage of occlusion
 - the value of SDF -> a "safe" angle seen from the eye



我们取点 P 为 shading point 往一方向打出一根光线,光线上的一点 a,有一个 SDF 值 SDF(a),也就是在 a 点以 SDF(a)为半径所做的球或圆内是安全的,不会碰到物体.

把 shading point 和面光源相连，所得到的安全角度越小，被遮蔽的可能越高，就可以认为：safe angle 越小，阴影越黑,越趋近于硬阴影；safe angle 够大就视为不被遮挡没有阴影,也就越趋近于软阴影。因此我们需要知道的应是如何从 ray marching 中求出 safe angle:

- During ray matching
 - Calculate the "safe" angle from the eye at every step
 - Keep the minimum
 - How to compute the angle?

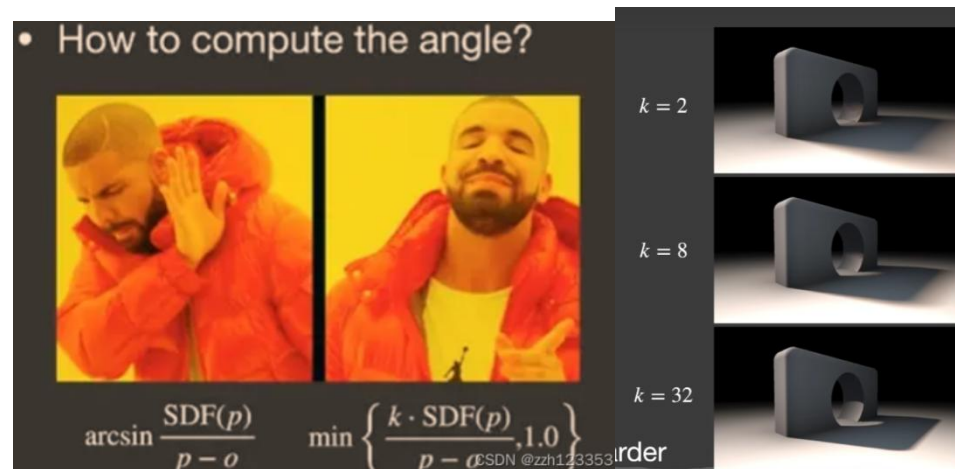


我们以 o 为起点,沿一个方向推进,仍然是 ray marching 的步骤,在 p1 点以 SDF(p1)进行推进,其余点也是一样,此处主要是为了求 safe angle,我们在起点 o 沿每个点的 sdf 为半径所形成的圆做切线,从而求出各个点的 safe angle,我们最后再取其中最小的角度作为总的 safe angle.

那么我们该怎么去计算这个角度?

从图我们可以知道,以 p_1 点为例,从 o 点到 p_1 的距离为斜边, $sdf(p_1)$ 是直角边,因此我们用 $\arcsin$ 就可以求出 $safe\ angle$ 了.

但是 $\arcsin$ 的计算量其实是十分大的,因此在 **shader 中我们不用反三角函数**.



只要 sdf 长度除以光线走过的距离乘一个 k 值,再限定到 1 以内,就能得到遮挡值或者说是 **visibility**,而 k 的大小是控制阴影的软硬程度.

我们从图中右半部分可以看出来,当 k 值越大时候,就越接近硬阴影的效果,也就是它限制了可能半影的区域:

k 越小,安全角度比较大,半影区域越大,越接近软阴影效果. k 越大,安全角度比较小,半影区域越小,越接近硬阴影效果.安全角度越大,阴影越软;安全角度越小,阴影越硬.

总结: SDF 是一个快速的高质量软阴影生成方法(比 $shadow\ map$ 快是忽略了 SDF 生成的时间),但是在存储上的消耗非常大,而且生成 SDF 的时间也要很久, SDF 是预计算,在有动态的物体的情况就得重新计算 SDF .

二、Environment lighting（环境光）

在 $games101$ 中我们知道,环境贴图就是在场景中任意一点往四周看去可看到的光照,将其记录在一张图上这就是环境光照,或者也可以叫做 $IBL(image-based\ lighting)$.这里我们认为看到的光照来自于无限远处,这也就是为什么用环境光照去渲染物体时会产生一种漂浮在空中的感觉,因为光照来自于无限远处.

通常我们用 $spherical\ map$ 和 $cube\ map$ 来存储环境光照.

如果已知环境光照,此时放置一个物体在场景中间,在不考虑遮挡时我们该如何去得到任何一物体上任何一 $shading\ part$ 的 $shading$ 值呢?

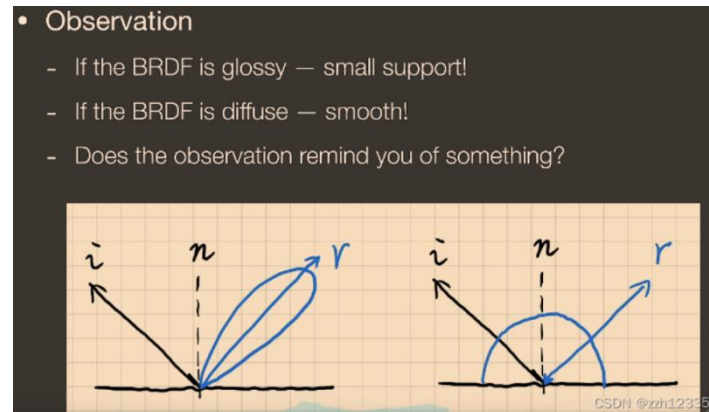
$$L_o(p, \omega_o) = \int_{\Omega^+} \underbrace{L_i(p, \omega_i)}_{IBL} \underbrace{f_r(p, \omega_i, \omega_o) \cos \theta_i}_{BRDF} \underbrace{V(p, \omega_i)}_{\text{Occlusion}} d\omega_i$$

在此先不考虑遮挡问题,通用的解法是使用蒙特卡洛积分去求解,但是采样太多才会导致结果比较接近,如果我们对每一个 $shading\ point$ 都做一遍蒙特卡洛,会导致效率很低下.

brdf 又分为两种情况:

brdf 为 glossy 时,覆盖在球面上的范围很小,也就是 small support(积分域).

brdf 为 diffuse 时,它会覆盖整个半球的区域,但是是 smooth 的,也就是值的变化不大,就算加上 cos 也是相对平滑的.



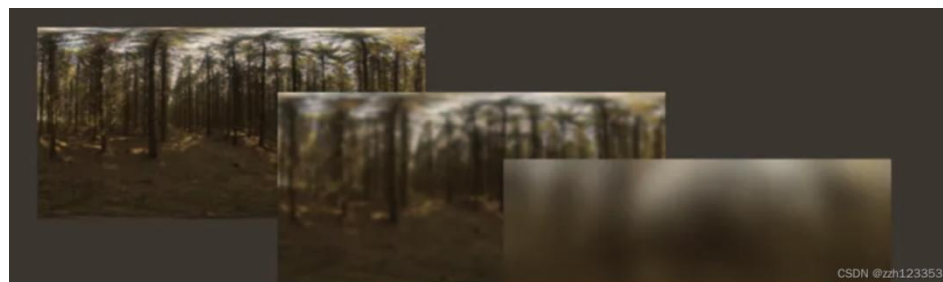
考虑到之前的积分近似公式,可以拆分成两部分,由于 brdf 项满足 accuracy condition,即 small support 或 smooth,从而我们将 rendering equation 的 lighting 项拆出让他变为:

- BRDF satisfies the accuracy condition in any case
 - We can safely take the lighting term out!

$$L_o(p, \omega_o) \approx \frac{\int_{\Omega_{f_r}} L_i(p, \omega_i) d\omega_i}{\int_{\Omega_{f_r}} d\omega_i} \cdot \int_{\Omega^+} f_r(p, \omega_i, \omega_o) \cos \theta_i d\omega_i$$

把 light 项拆分出来,然后将 brdf 范围内的 lighting 积分起来并进行 normalize,其实就是将 IBL 这张图给模糊了.

模糊就是在任何一点上取周围一片范围求出范围内的平均值并将平均值写回这个点上,主语滤波的核取多大取决于 BRDF 占多大,BRDF 的区域越大,最后取得的图也就越模糊.

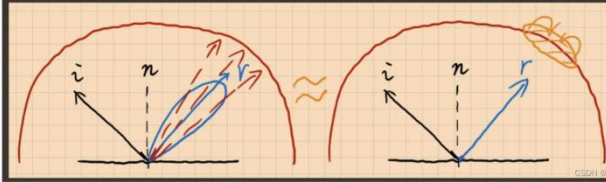


而这些模糊过的图是在我们进行 rendering 之前生成的,也就是 pre-filtering, Prefiltering 就是提前把滤波环境光生成,提前将不同卷积核的滤波核的环境光生成一系列模糊过的图(和 mipmap 相似),当我们需要时进行查询即可,其他尺寸的图则可以经过这些已生成的通过三线性插值得到.

之前提过,拆分就是为了做一个 Pre-filtering,那么做 pre-filtering 是为了干什么?

The Split Sum: 1st Stage

- Then query the pre-filtered environment lighting at the r (mirror reflected) direction!



左图为 brdf 求 shading point 值时,我们要以一定立体角的范围内进行采样再加权平均从而求出 shading point 的值.

右图为我们从镜面反射方向望去在 pre-filtering 的图上进行查询,由于图上任何一点都是周围范围内的加权平均值,因此在镜面反射方向上进行一次查询就等价于左图的操作,并且不需要采样,速度更快.

到此我们解决了拆分后的前半部分积分采样的问题,那么接下来我们处理 BRDF 项采样的问题:

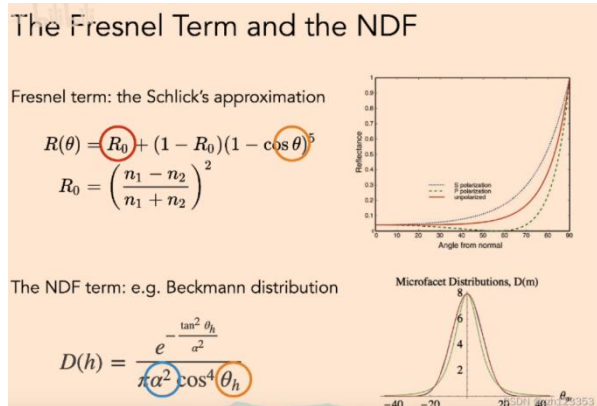
$$L_o(p, \omega_o) \approx \frac{\int_{\Omega_{fr}} L_i(p, \omega_i) d\omega_i}{\int_{\Omega_{fr}} d\omega_i} \cdot \int_{\Omega^+} f_r(p, \omega_i, \omega_o) \cos \theta_i d\omega_i$$

接下来讲的方法并不是最优方法,现如今已经有更简单方便的方法了,但是本课我们主要是为了学习方法背后的思想.

我们仍然可以用预计算来解决后半部分积分采样的问题,但是预计算的话我们需要将参数的所有可能性均考虑进去,但是比较多,包括 roughness、color 等。考虑所有参数的话我们需要打印出一张五维或者更高的表格,这样会拥有爆炸的存储量,因此我们需要想办法降低维度,也就是减少参数量从而实现预计算.

$$f(i, o) = \frac{F(i, h)G(i, o, h)D(h)}{4(n, i)(n, o)}$$

在 microfacet brdf 中,考虑的是菲涅尔项、阴影项以及法线项,由于此时暂时不考虑阴影,此处需要关注的是 Fresnel term 和 distribution of normals。



Fresnel term 可以近似成一个基础反射率 R_0 和入射角度的指数函数

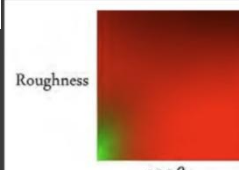
法线分布函数 (NDF) 是一个一维的分布, 其中有两个变量, 一个变量定义是 **diffuse** 还是 **gloosy**, 另一个是 **half vector** 和法线中间的夹角, 可以近似成入射角度相关的数, 这样就变成了 3 维的预计算。

(PS:在这里我们认为反射角,入射角,half vector 可以用一个角 θ 代替).

至此我们有了三个变量:基础反射率 r_0 , roughness α 和角度 θ , 三维的预计算仍然是一个存储量爆炸的结果, 因此我们还要想办法减少参数量。

所以我们通过将 Schlick 近似带入后半部分的积分中:

基础反射 R_0 被拆出积分式, 需要预计算的两个量就只有 roughness α 和角度 θ , 可以将预计算结果绘制成一张纹理, 在使用时进行查询即可。

$$\int_{\Omega^+} f_r(p, \omega_i, \omega_o) \cos \theta_i d\omega_i \approx R_0 \int_{\Omega^+} \frac{f_r}{F} (1 - (1 - \cos \theta_i)^5) \cos \theta_i d\omega_i + \int_{\Omega^+} \frac{f_r}{F} (1 - \cos \theta_i)^5 \cos \theta_i d\omega_i$$


我们可以看到,最后产生的结果是十分满意的:

那么在有了环境光照情况下如何去得到物体被环境光照射下生成的阴影呢?

严格意义上来讲,这是不可能完成的事,因为以目前的技术来说是很难实现的,要从两个考虑角度来说:

1.many light 问题:我们把环境光理解为很多个小的光源,这种情况下去生成阴影的话,需要在每个小光源下生成 **shadow map**,因此会生成线性于光源数量的 **shadow map**,这是十分高昂的代价。

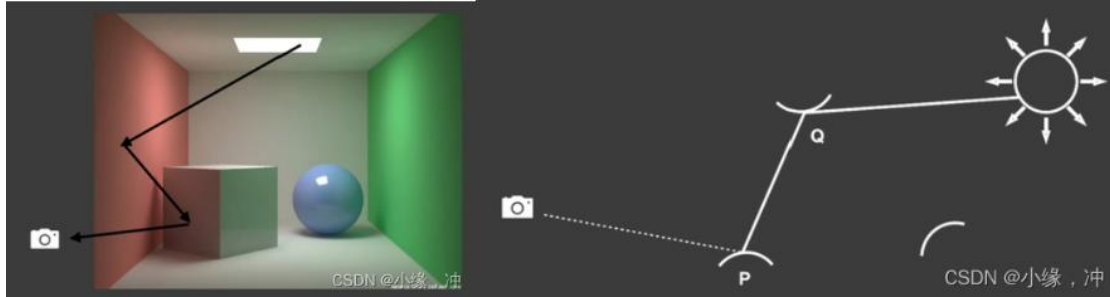
2.sampling 问题:在任何一个 **Shading point** 上已知来自正半球方向的光照去接 **rendering equation**,最简单的方法是采样空间中各方向上的不同光照,可以做重要性采样,虽然做了重要性采样但仍需要大量的样本,因为最困难的是 **visibility term**. 由于 **Shading point** 不同方向上的遮挡是不相同的,我们可以对环境光照进行重要性采样,但一个 **Shading point** 周围的 **visibility** 项是未知的,因此我们只能盲目的去采样(我个人对盲目采样的理解是,为了确保准确性需要对 **Shading point** 各个方向的遮挡进行采样,因此仍然会生成大量的样本). 我们也无法提取出 **visibility** 项,因为如果是 **glossy brdf**,他是一个高频的,且 **Lighting** 项的积分域是整个半球,因此并不满足 **smooth** 或 **small support**,因此无法提取出 **visibility** 项。

在工业界中,我们通常以环境光中最亮的那个作为主要光源,也就是太阳,只生成太阳为光源的 **shadow**。

二、Global Illumination（全局光照）

a. 全局光照就是直接光照加上间接光照,通过模拟光线的传播路径,将物体反射的间接光纳入计算,从而提高结果真实感的一种渲染技术

在实时渲染中,全局光照是比直接光照多一次 **bounce** 的间接光照.如下图左中,光线弹射两次,先打到红色的墙壁上,在达到 **box** 面上,最后被 **camera/eye** 看到,这就是在 **rtr** 中要解决的所谓的"全局光照".



图右,从 **camera** 出发打出一条光线到点 **P**, **P** 点又往场景中不同的方向发射光线,如果打到光源则表示接受的是直接光照,如果没打到光源而是打到了点 **Q**,那么我们认为 **P** 点接收到的光照是从 **Q** 点反射到 **P** 点的 **Radiance**,也就是 **Q** 点接收到的直接光照所反射出的光照打到 **P** 点上。点 **Q** 被当作次级光源(**Secondary Light Source**)用自身反射的光照来照亮其他物体。

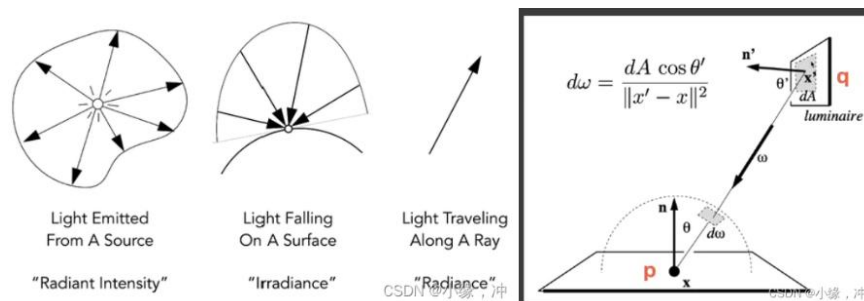
要得到 **p** 点的光照结果,我们需要知道:哪些东西是次级光源和每个次级光源对 **p** 点光照的贡献

b. Reflective Shadow Map(RSM)

原理:处理完直接光照以后,在 **Shadow Map** 中被照亮的点直接当作次级光源使用,并影响附近的点,当作间接光照照亮它们。

哪些东西是次级光源?

使用 **shadow map**, **shadow map** 上每一个像素可以看成是一个小 **surface patch**,是一个次级光源,对于不同的 **P** 点来说出射方向是不知道的,无法计算 **P** 点的 **shading** 的,为了不依赖于观察方向,我们假设所有的反射物(次级光源)都是 **diffuse** 的。



Radiant Energy:光能-->一个区域内光子能量的总和,用 **Q** 来表示,单位是焦耳(J).

Radiant Flux:光通量-->在单位时间内穿过单位截面积的光能.

Radiant Intensity: 一个单位立体角上对应的能量。(单位立体角上的光通量)

Irradiance: 在一个单位面积下对应的能量。(单位面积上的光通量)

Radiance: 一个单位立体角下单位面积上的能量。(单位立体角上通过单位投影面积的光通量)

图右, q 是一个 patch 去照亮点 p , 将其转化成面积的积分。(把立体角的积分变成了对 light 区域面积的积分), 对于每个次级光源点来说, 由于我们假设它的 brdf 是 diffuse 的, 因此次级光源的 f_r 积分后是个常数。 v 表示次级光源对 p 点的可见性, 选择忽略。

$$L_o(p, \omega_o) = \int_{\Omega_{\text{patch}}} L_i(p, \omega_i) V(p, \omega_i) f_r(p, \omega_i, \omega_o) \cos \theta_i d\omega_i$$

$$= \int_{A_{\text{patch}}} L_i(q \rightarrow p) V(p, \omega_i) f_r(p, q \rightarrow p, \omega_o) \frac{\cos \theta_p \cos \theta_q}{\|q - p\|^2} dA$$

- For a diffuse reflective patch
 - $f_r = \rho / \pi$
 - $L_i = f_r \cdot \frac{\Phi}{dA}$ (Φ is the incident flux or energy)

CSDN @小缘, 冲

此时我们要求解 $Li(q \rightarrow p)$, 它即为 q 点沿着某一方向的能量值, 由于默认 q 为 diffuse 的, 所以其 brdf 值为常数, $Li(q \rightarrow p)$ 即为其入射的 irradiance (单位面积的能量值), 其值为总能量 flux 值除以该小部分面积 dA 。

$$E_p(x, n) = \Phi_p \frac{\max\{0, \langle n_p | x - x_p \rangle\} \max\{0, \langle n | x_p - x \rangle\}}{\|x - x_p\|^4} \quad (1)$$

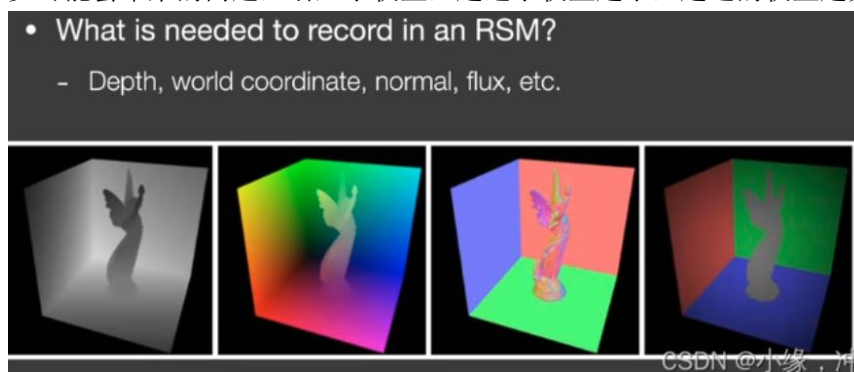
CSDN @小缘, 冲

由于可见性、方向性以及距离的不同, 对于某一个着色点, 认为 shadow map 上所有的 pixel 不可能都有贡献: 因为远处的次级光源贡献很少, 通常只要找距离足够近的次级光源就行了。

加速方法: 我们认为, 对着色点 x 影响大的间接光源在 shadow map 中一定也是接近的。

先获取着色点 x 在 shadow map 中的投影位置

在该位置附近采样间接光源, 多选取一点离着色点近的 VPL, 并且为了弥补越往外采样数越少可能会带来的问题, 引入了权重, 越近了权重越小, 越远的权重越大。



优点: 好实现。RSM 效果通常应用于游戏中手电筒的次级光照。

缺点:

性能随着直接光源数的增加而降低(因为需要计算更多的 shadow map)

对于间接光照, 没有做可见性检查, 无法判断次级光源和当前 Shading Point 的可见性关系

有许多假设: 反射物需要是 diffuse 等

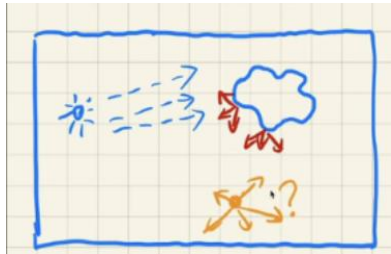
需要在质量和采样率上做一个平衡

c. Light Propagation Volumes (LPV)

把场景分为 3D 的网格, 让所有次级光源进行传播迭代。用直接光源找出所有次级光源以后,

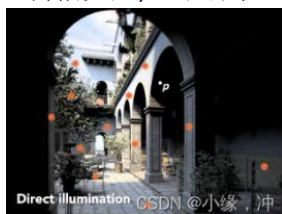
将他们的方向,光照等信息记录在格子内,每次迭代将它们传播到周围的格子;迭代若干次以后,每个 **Shading Point** 根据当前处于的格子拿到次级光照信息。

优点: 快,质量好,解决一部分 **RSM** 的问题。

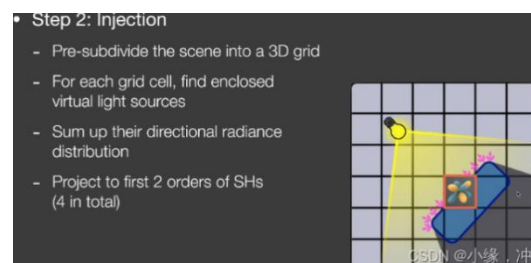


如果我们能获得任何一个 **Shading point** 上来自四周的 **radiance** 的话,就可以立刻得到其间接光照,我们假设光在传播过程中,**radiance** 是 **uniform** 的。

- 1.找出接收直接光照的点
- 2.把这些点注入(**inject**)到 **3D** 网格中作为间接光照(虚拟光源)的传播起点.
- 3.在 **3D** 网格中传播 **radiance**
- 4.传播完后,渲染场景

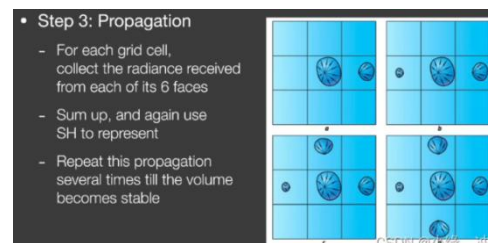


与 **RSM** 一样,首先通过 **Shadow Map** 找出接受直接光照的表面或物体(光源数量可以通过采样一些进行简化从而降低次级光源数量,最后获得一系列虚拟光源)



把场景划分为若干个 **3D** 网格(体素),把虚拟光源注入到其对应的格子内,一个格子内可能包含许多不同朝向的虚拟光源,把格子内所有虚拟光源的不同朝向的 **radiance** 算出来并 **sum up** 从而得到一个往四面八方的 **radiance**。

由于是在空间上的分布,也就可以看作是球面函数,自然可以用 **SH** 来表示(工业界用两阶 **SH** 就可以表示各个方向上的 **radiance** 初始值)



3D 网格,因此可以向六个面进行传播(上下左右前后),由于 **radiance** 是沿直线传播的,我们认为 **radiance** 是从网格中心往不同方向进行传播的,穿过哪个表面就往哪个方向传播,每个格子计

算收到的 radiance,并用 SH 表示,迭代四五次之后,场景中各 voxel 的 radiance 趋于稳定。

• Step 4: Rendering

- For any shading point, find the grid cell it is located in
- Grab the incident radiance in the grid cell (from all directions)
- Shade

CSDN @小缘,冲

对于任意的 shading point, 找到他所在的网格获得所在网格中所有方向的 Radicae 渲染。



LPV 提供的这种实现全局光照的方法不需要任何的预计算,因此可以支持各种变化的场景,做到真正的“实时”。但体素划分精度不足时,会出现漏光的现象

解决漏光现象: 需要我们划分的格子足够小,这样会导致存储量增多。

LPV 会假设次级光源是 diffuse 的吗? 一般只要用了 SH 都会假设 diffuse

体素划分一般多大比较合适? 比场景分辨率少一个数量级

这是预计算吗? 不是,这是实时的,在任意一帧都要去做这个计算。

传播过程中会一直不稳定传播吗? 不会,没有外界影响的话最后肯定是会收敛的。

d. Voxel Global Illumination (VXGI)

对场景构建体素网格,在第一个 Pass 中存储所有次级光源的贡献和法线信息,并更新 Hierarchy 层级结构; Pass2 中对每个次级光源射出一个圆锥形的射线,为其他 Shading Point 做贡献, diffuse 类型的用多个圆锥射线来覆盖亮半球。

• Two main differences with RSM

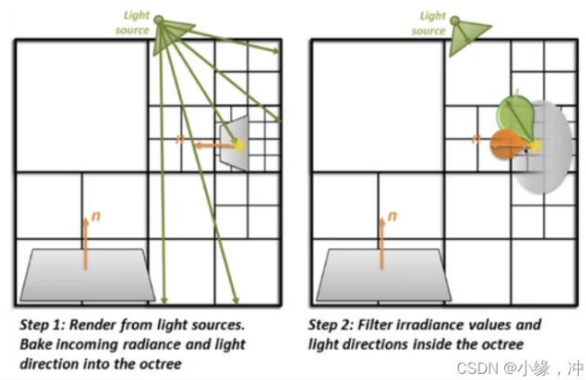
- Directly illuminated pixels -> (hierarchical) voxels
- Sampling on RSM -> tracing reflected cones in 3D (Note the inaccuracy in sampling RSM)



RSM 中次级光源是像素中所包含的微小表面,这些表面是根据 Shadow Map 来划分的。VXGI 把场景完全离散化成了一系列微小的格子。从 camera 出发,就像有一个 Camera Ray 打到每一个 pixel 上,根据 pixel 上代表的物体材质做出不同的操作,如果是 glossy 则打出一个锥形区域,diffuse 则打出若干个锥形区域,打出的锥形区域与场景中一些已经存在的 voxel 相交。

Pass1: Light pass

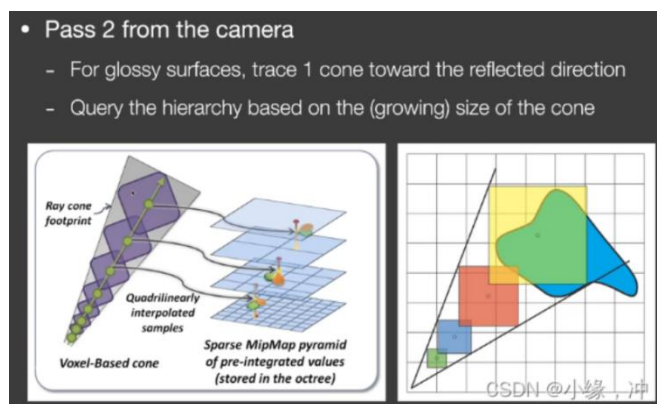
记录的是直接光源从哪些范围来(绿色部分),记录各个反射表面的法线(橙色部分),通过输入方向和法线范围两个信息然后通过表面的材质,来准确的算出出射的分布。



Pass 2 :Camera pass

对于任何一个像素，知道了 Camera Ray 的方向。

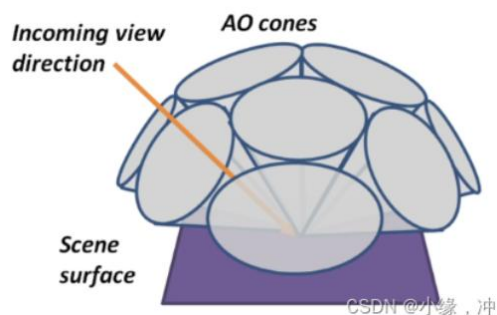
Glossy: 向反射方向追踪出一个锥形(cone)区域:



基于追踪出的圆锥面的大小，对格子的层级进行查询，就是对于场景中的所有体素都要判断是不是与这个锥形相交，如果相交的话就要把对于这个点的间接光照的贡献算出来。

(mipmap)

diffuse: VXGI 通常会将出射方向看做若干圆锥，而忽略锥与锥之间的间隙。



LPV 是把所有的次级光源发出的 Radiance 传播到了场景中的所有位置，只需要做一次从而让场景每个 Voxel 都有自己的 radiance，但是由于 LPV 使用的 3D 网格特性，并且采用了 SH 进行表示和压缩，因此结果并不准确，而且由于使用了 SH 因此只能考虑 diffuse 的,但是速度是很快的。

VXGI 把场景的次级光源记录为一个层次结构，对于一个 Shading Point，我们要去通过 Corn Tracing 找到哪些次级光源能够照亮这个点。

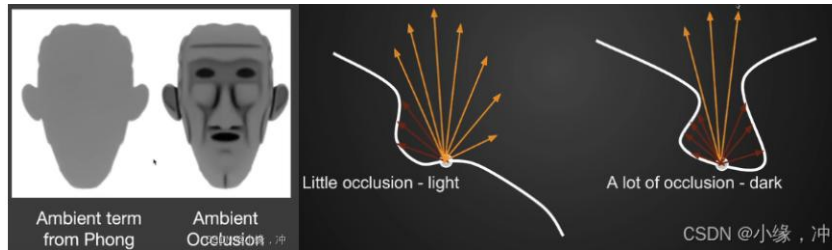
e. Screen Space Ambient Occlusion 屏幕空间环境光遮蔽

AO (环境光遮蔽): 在场景中加入 AO 信息，让物体与物体之间的相对位置表示的更明显，

让整个画面更具立体感。AO 是一个对于全局光照的近似。



在计算全局光照过程中，由于我们无法在屏幕空间中直接获得间接光照，所以一般会假设间接光照强度为一个定值，但并不是每个方向都可能接收到间接光照(incident lighting)，也就是不同位置的 Visibility 是不同的，假设物体是 Diffuse 的。



左边是任何一个 shading point 上来自任何方向的间接光照(incident lighting)是一个常数.右边是我们考虑了任何一个 shading point 上不同方向的 visibility 后得到的结果。图中红色部分表示被遮挡,黄色部分表示未被遮挡,我们可以明显的得出一个结论,左边的 Shading point 要比右边的亮一点。

通过使用约等式（RTR 近似方程）将渲染方程中 visibility 项提出后，得到：

• Separating the visibility term

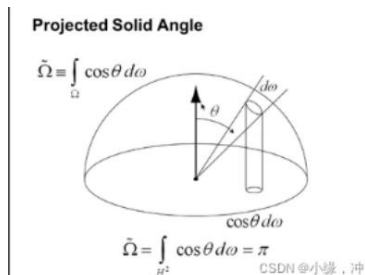
$$L_o^{\text{indir}}(\mathbf{p}, \omega_o) \approx \frac{\int_{\Omega^+} V(\mathbf{p}, \omega_i) \cos \theta_i d\omega_i}{\int_{\Omega^+} \cos \theta_i d\omega_i} \int_{\Omega^+} L_i^{\text{indir}}(\mathbf{p}, \omega_i) f_r(\mathbf{p}, \omega_i, \omega_o) \cos \theta_i d\omega_i$$

$\square \triangleq k_A = \frac{\int_{\Omega^+} V(\mathbf{p}, \omega_i) \cos \theta_i d\omega_i}{\pi}$
 $\square = L_i^{\text{indir}}(\mathbf{p}) \cdot \frac{\rho}{\pi} \cdot \pi = L_i^{\text{indir}}(\mathbf{p}) \cdot \rho$

(the weight-averaged visibility
 $\bar{V}$ from all directions)

CSDN @小缘, 冲

不是对 $d\omega_i$ 进行积分,而是对 $\cos\theta d\omega$ 进行积分,采用投影立体角。立体角是单位球上的一个面积,关于 $\cos$ 中的 θ ,我们认为 θ 从北极开始到南极是 180 度,那么立体角 * $\cos\theta$ 是我们把单位球上的面积投影到了单位圆上。



间接光照是常数—Li 为常数

BRDF 是 Diffuse 的—BRDF (fr) 是常数 ρ/π

因此这两项可以直接从积分里面拿出来,最后 Rendering Equation 就成了下面的形式,也就是算 Visibility 的积分。

$$\begin{aligned} L_o(p, \omega_o) &= \int_{\Omega^+} L_i(p, \omega_i) f_r(p, \omega_i, \omega_o) V(p, \omega_i) \cos \theta_i d\omega_i \\ &= \frac{\rho}{\pi} \cdot L_i(p) \cdot \int_{\Omega^+} V(p, \omega_i) \cos \theta_i d\omega_i \end{aligned}$$

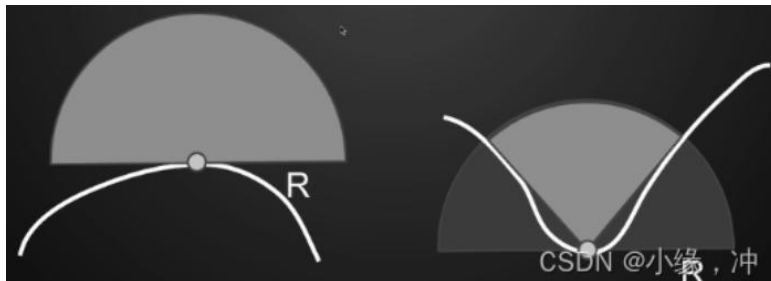
CSDN @小缘, 冲

重点: 算加权平均的 visibility:

我们需要在 shading point 往不同方向 trace 判定究竟有多少方向被挡,这样说其实是不准确的,我们以一个极端的例子来看,在一个封闭的屋子里,不管你从哪一个 shading point 去 trace 哪个方向,不论 trace 的光线会不会打到周围的物体,我们这跟光线不会出去这个封闭的屋子,也就是最后的结果只能是被遮挡住,所得到的 Ambient Occlusion 只会是 0.

这是因为,反射光肯定是在有限的距离里反射过来的,也就是间接光照是从一定范围内来的,不可能是从无限远处,因此如果我们做 tracing 肯定也是在一定范围内的,这样就解决了 tracing 无限远的话一定会被遮挡住这个问题。

但是出现了新问题,就是超出这个范围的光照就被我们忽略了,也就是我们忽略了那些在距离外的间接光照,如果我们限制范围太大,那么所有东西都有可能遮挡住,如果范围太小,我们会忽略大部分的 incident lighting. 因此我们限制在一定范围内,这是一个 trade off,通常会选一个合适的范围,也就是找一个合适的球的半径-R.



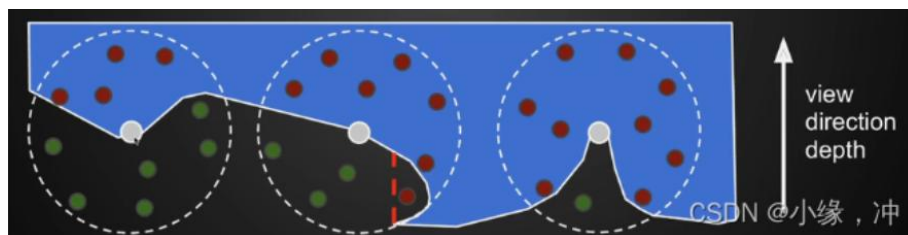
工业界的方法:

首先从相机出发,得到屏幕空间的深度信息,写入 z-buffer

对屏幕空间的任意一个像素(着色点),在以它为中心、R 为半径的球体范围内随机寻找数个采样点,判断其可见性

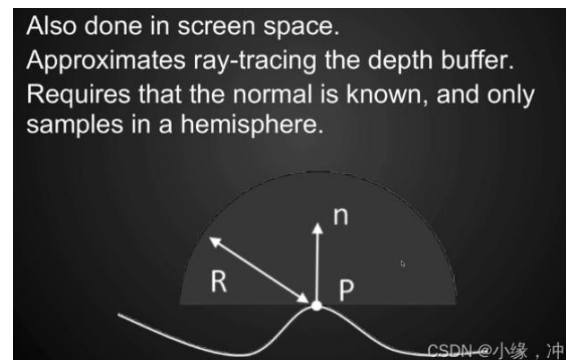
若该采样点的深度大于它在屏幕空间对应的深度,则认为该点不可见,记可见性为 0 (下图绿点)

计算可见性为 1 的采样点的占比,作为当前像素的 visibility 值



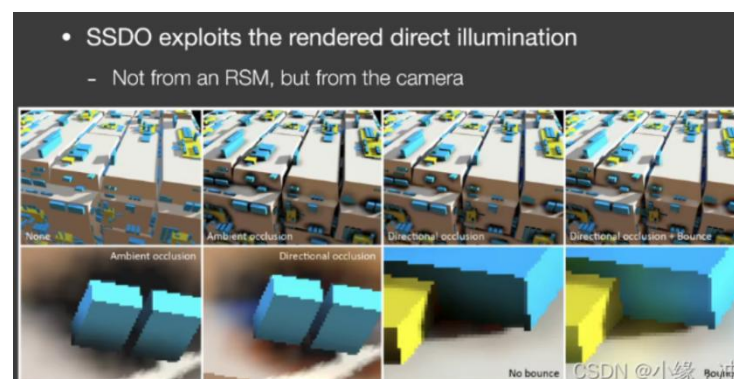
上图中蓝色区域部分表示的是场景物体,也就是说蓝色区域外的绿点是可以被 shading point 直接看到的.图的一个红点出现了问题,因为我们从 camera 看过去,这个点比所圈部分的深度深,因此虽然这个红点是能够被 Shading Point 看到的,但是由于是从 camera 出发,这里被判断为了看不到。

HBAO (Horizon Based ambient occlusion): 当有了场景 normal 之后,我们就知道去采样哪半球,就可以只去算半球的情况了,同时也可以对不同的方向来加权(靠近中间大,靠近两边小),使用多次光线步进找到着色点切平面与遮挡物采样点形成的最大角度,再通过这个角度来计算物体与物体间的遮挡程度,从而完成对环境光遮蔽的近似。



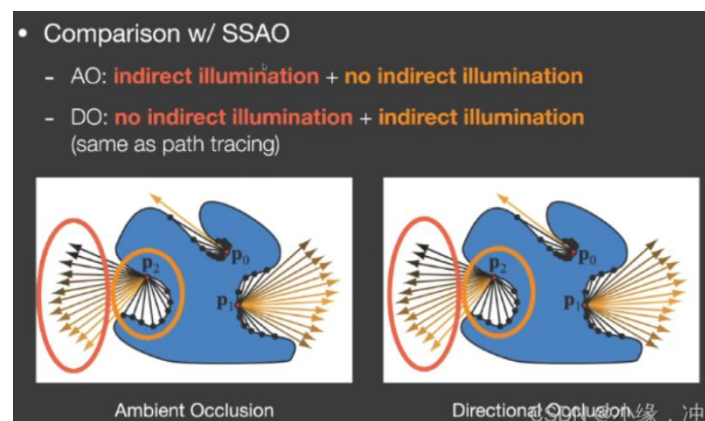
f. Screen space Direction Occlusion (SSDO)

SSDO 是对于 SSAO 的改进算法,类似于 SSAO,在周围随机找点,看 Z-Buffer 的遮挡关系;与 SSAO 相反的是,被遮挡的点,从 View 方向找到深度最小的点,利用那一点对当前点做出贡献;否则,可以利用环境光做出贡献。



AO 能够产生变暗的效果使得物体相对感更强烈,DO 蓝色面上会接收到一点黄光,黄色面上也会有一点蓝光。SSDO 只能解决一个很小范围内的全局光照。

在 SSDO 中,我们要用直接光照的信息,但不是从 RSM 中获得,而是从 screen space 中得到。



在 AO 中我们认为红色的框里能接收间接光照，黄色框里无法接收间接光照，然后求出加权平均的 visibility 值,也就是假设间接光照是从比较远的地方来的。

在 DO 中,我们认为红色框里接收的是直接光照,而黄色框里才是接收到的间接光照.因为间接光照一定要打在某个物体上再反射到 P 点。假设间接光照是从比较近的反射物来的。

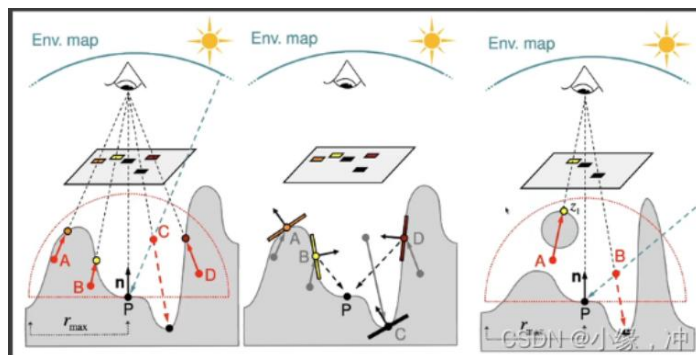
$$L_o^{\text{dir}}(p, \omega_o) = \int_{\Omega^+} L_i^{\text{dir}}(p, \omega_i) f_r(p, \omega_i, \omega_o) \cos \theta_i d\omega_i$$

$$L_o^{\text{indir}}(p, \omega_o) = \int_{\Omega^+} L_i^{\text{indir}}(p, \omega_i) f_r(p, \omega_i, \omega_o) \cos \theta_i d\omega_i$$

CSDN @小缘, 冲

当 $V=1$ 时是直接光照，而 DO 的计算是计算间接光照的，因此这个我们完全不用去计算与考虑，当 $V=0$ 时是间接光照。

考虑点 P 法线部分的半球，判断从 P 点往 A、B、C、D 四个方向看会不会被挡住，这里 A/B/D 这三个点的深度比从 camera 看去的最小深度深,也就是说 PA,PB,PD 方向会被物体挡住,因此会为 P 点提供间接光照。



右图，A 点虽然会被 camera 看不到，但是 AP 之间是不会挡住的,实际上 A 点需要提供间接光照给 P 点,但在 SSDO 算法中则不提供。

g. Screen space Reflection (SSR)

SSR 本质上在屏幕空间做光线追踪，只需要屏幕空间中已有的信息，也就是从 camera 看去场景的得到的这样一层“壳”。考虑任何光线(不单单是反射光)与场景中这层壳去做求交。找到交点后,算出对 shading point 的贡献值。

在屏幕空间下，从 View 方向看去，对每个 Shading Point 做简单的 Ray Tracing，来获得当前点的次级光照信息。



镜面反射：假设场景中已经渲染出来了上面的部分，对于地面还没有进行渲染，如何把反射的信息加进去。对于任何一个像素：

- 1.知道 shading point 的观察方向后,可以得出其反射方向
- 2.从 Shading point 点沿着反射方向延长找到与屏幕的壳的交点
- 3.将交点的颜色作为反射的颜色记录到 shading point。

specular 反射:

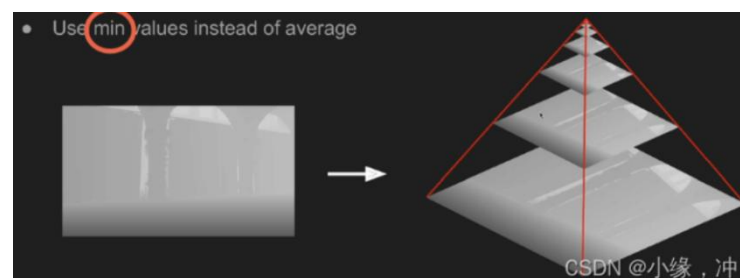
- 1.我们有一个还没有进行反射的场景,如 shaded scene 上的地面还并没有得到反射的结果得到图中的 normal 信息和深度信息
- 2.进行 SSR,我们想要的是对于地面的每一个 pixel,我们都想计算出其反射到场景中的得到的值是多少
- 3.得到反射的值后,将结果加到场景中去,也就是在地面上的黄点反射到场景的壳上的绿色,进行求交算到的结果加入到黄点中
- 4.得到一个镜面反射的效果

求反射光与场景的相交: Linear Raymarch:

让光线按一定步长向前迈进,同时判断是否与物体产生相交。沿着反射方向以一个固定的步长逐步前进,并将每次停止时的深度与壳的深度进行比较,如果浅于壳,则继续前进,比壳深,则停止求交,也就是我们用深度来进行可见性判断。质量取决于步长的大小,步长小越精准,同时计算量也越大,因此步长太大太小都不行,在没有 SDF 的情况下,步长只能是一个定值。

动态决定步长: Hierarchical ray trace

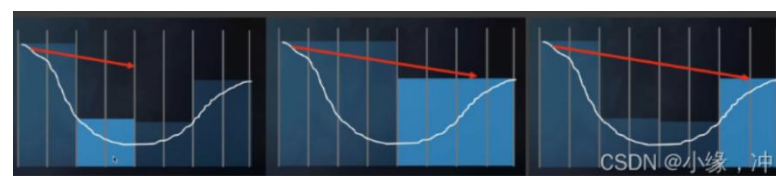
类似 Mipmap, 对深度缓存中的屏幕深度生成一个层级纹理, 这个过程并不是像 Mipmap 那种做双线性插值四个像素的深度平均, 而是取四个像素的最小深度直接写入缓冲。



在 3D 空间做光线追踪时,为了加速光线与场景求交, 我们通常会做一个加速的层次结构 (BVH/KD-tree)

如果我们用最小值操作的 mip-map 会得到一个保守逻辑:如果一根光线与 mip-map 中的上层结点不相交,那他肯定也不会与这个结点的子节点相交。

走格子(像素) 2-4-8 等递增, 判断有交点之后, 退回上一级再判断。但是做不了准确的起点不在 2 的 K 次方上的深度的查询。



缺点: SSR 仍然有屏幕遮挡的问题, 会丢失一些反射中的模型信息 (因为只有屏幕中的信息); 同样的, 因为只有屏幕空间的信息, 在边缘外的会被截断 (可以利用距离反射点越远, 越虚化来做处理)。



计算 shading:

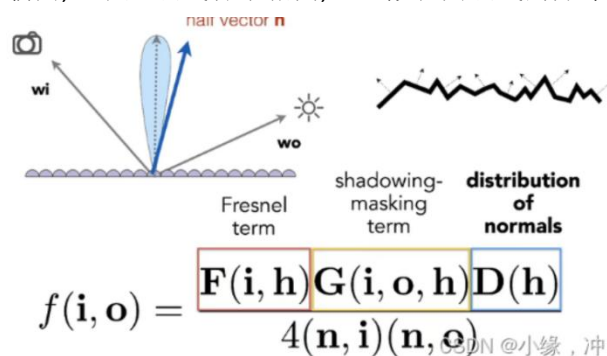
SSR 计算着色的方法几乎与路径追踪一模一样, 对于任何一个 shading point, 看到的 radiance 就是对半球进行积分, 如果是 specular 材质则 cast 一次, 相当于光线打到物体的哪里, 就用它所发出的 radiance 就可以。glossy 材质就用蒙特卡洛重要性采样多次计算 (这里需要假设次级反射物为 diffuse)

SSR 不用考虑光线的平方衰减, 并且可以保证交点的可见性, 这是因为 SSR 的采样对象是 brdf 本身, 而只有在对光源采样的时候才要考虑衰减和遮挡。

三、Physically-Based Materials and PBR (Physically-Based Rendering) 基于物理的材料以及渲染

a. Microfacet models 微表面模型

我们认为在宏观上看上去是平的, 但是在微观上看去会看到各种各样的微表面, 这些微表面的朝向, 也就是法线各不相同, 这些微表面法线的分布导致的渲染出的结果各不相同。



F 项: 菲涅尔项

表示观察角度与反射的关系, 从一个角度看会有多少的能量被反射。

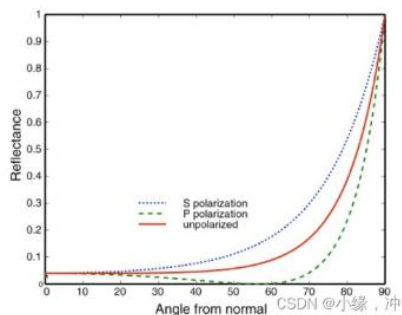
有多少能量被反射取决于入射光的角度, 当入射方向接近 grazing angle 掠射角度的时候, 光线是被反射的最多的, 也就是当你的入射方向与法线几乎垂直时候, 反射的 radiance 是最多的。

Reflectance depends on incident angle (and polarization of light)

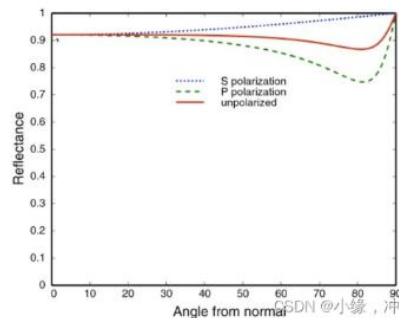


对于绝缘体反射率与角度（左），入射方向与法线几乎垂直反射最多，导体与绝缘体不同，部分会出现反常现象。

Fresnel Term (Dielectric, $\eta=1.5$)



Fresnel Term (Conductor)



菲涅尔项公式比较复杂,因为要考虑不同介质,各自的折射率和入射角折射角:

$$R_s = \left| \frac{n_1 \cos \theta_i - n_2 \cos \theta_t}{n_1 \cos \theta_i + n_2 \cos \theta_t} \right|^2 = \left| \frac{n_1 \cos \theta_i - n_2 \sqrt{1 - \left(\frac{n_1}{n_2} \sin \theta_i\right)^2}}{n_1 \cos \theta_i + n_2 \sqrt{1 - \left(\frac{n_1}{n_2} \sin \theta_i\right)^2}} \right|^2$$

$$R_p = \left| \frac{n_1 \cos \theta_t - n_2 \cos \theta_i}{n_1 \cos \theta_t + n_2 \cos \theta_i} \right|^2 = \left| \frac{n_1 \sqrt{1 - \left(\frac{n_1}{n_2} \sin \theta_i\right)^2} - n_2 \cos \theta_i}{n_1 \sqrt{1 - \left(\frac{n_1}{n_2} \sin \theta_i\right)^2} + n_2 \cos \theta_i} \right|^2$$

$$R_{\text{eff}} = \frac{1}{2} (R_s + R_p).$$

使用一个简单的近似: Schlick's approximation

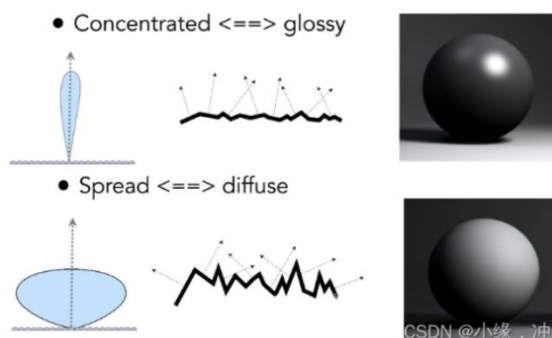
$$R(\theta) = R_0 + (1 - R_0)(1 - \cos \theta)^5$$

$$R_0 = \left(\frac{n_1 - n_2}{n_1 + n_2} \right)^2$$

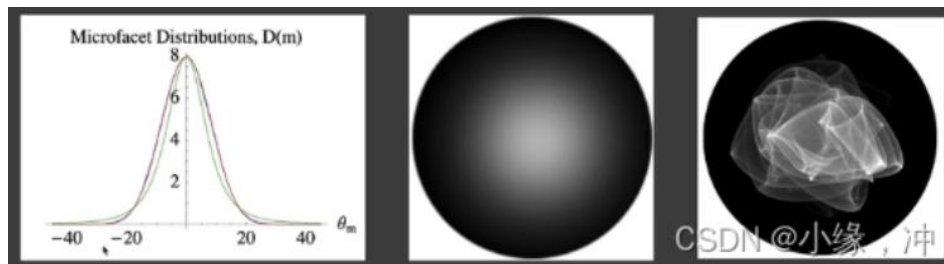
当 $\theta \rightarrow 90^\circ$, $\cos \theta = 0$, 则 $R(\theta) = 1$; 当 $\theta \rightarrow 0^\circ$, $\cos \theta = 1$, 则 $R(\theta) = R_0$; 其中 R_0 (基础反射率) 取决于物体。

D 项: 微表面的法线分布

表示不同微表面朝向的法线分布, 当朝向比较集中的时候会得到 Glossy 的结果, 如果朝向特别集中指向时认为是 specular 的, 当分布杂乱无章时候, 认为表面也非常复杂, 得到的结果类似 diffuse。



NDF: Normal Distribution Function, 把为表面法线分布的函数定义为 Normal Distribution Function (NDF)。



Beckmann 模型

描述法线分布, 是关于法线方向的函数, 即 H 的分布。 h 是半球上任一方向, 为了描述这一方向对应的值是多少我们需要 NDF。函数有点像高斯函数。

$$D(h) = \frac{e^{-\frac{\tan^2 \theta_h}{\alpha^2}}}{\pi \alpha^2 \cos^4 \theta_h}$$

可以描述不同粗糙程度的表面, 不同粗糙程度的意思是 NDF 中 lobe 是集中在一个点上, 还是分布的比较开。只与 θ 有关, 与 ϕ 无关, 表述的是各向同性的结果。

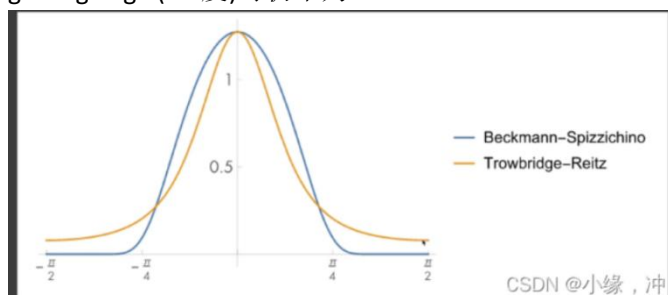
α 描述的是法线粗糙程度, 粗糙程度这个值越小, 表面就越光滑

θ_h : 微表面半程向量法线与宏观表面法线 $(0, 0, 1)$ 方向 n 的夹角

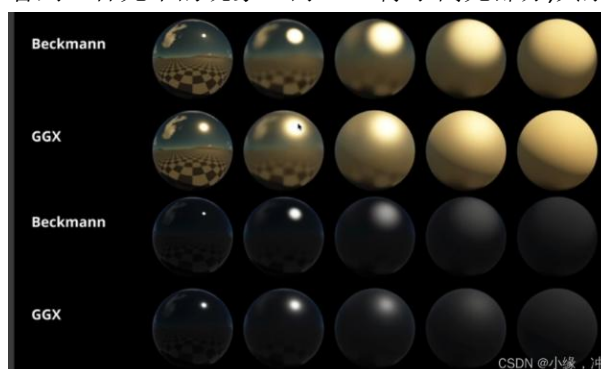
GGX 模型 (TR 模型)

Beckmann 模型的 NDF 曲线与 GGX 模型的 NDF 曲线相比有一个明显的特点: Long tail 长尾性质。

GGX 模型会很快衰减, 但是衰减到一定程度时候衰减速度会变慢, 可以看到即使到了 grazing angle (90 度) 时仍不为 0。

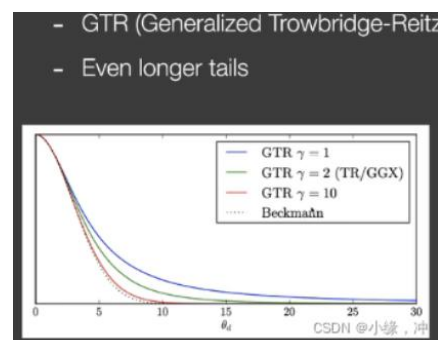


Beckmann 的高光会逐渐消失, 而 GGX 的高光会减少而不会消失, 这就意味着高光的周围我们看到一种光晕的现象。而 GGX 除了高光部分, 其余部分会像 Diffuse。



相同的粗糙程度下 GGX 的效果更加自然,因为 long tail 性质导致高光到非高光有一个柔和的过渡状态。

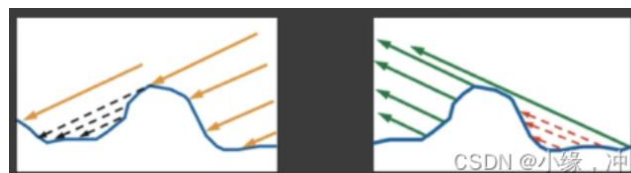
GTR (Generalized Trowbridge-Reitz)



多了参数 γ ，根据 γ 不同可以调节拖尾长度，包括了本身 GGX，当 $\gamma=2$ 时候就是 GGX，当 γ 超过 10 会接近 Beckmann；具有更长的拖尾。

G 项: Shadowing-Masking-Geometry Term

解决微表面之间的自遮挡问题，尤其是在角度接近 grazing angle 时。

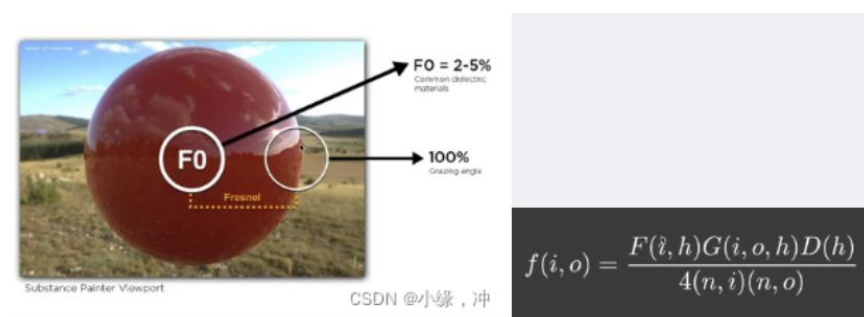


由于在微观上有不同的微表面,因此虚线部分本该入射的光被遮挡了，同时，往外看时也被遮挡。

左-从 light 出发发生的微表面遮挡现象叫做 Shadowing

右-从 eye 出发发生的微表面遮挡现象被称为 Masking

引入 G 项就是为了考虑由于遮挡产生的 darkening 现象(由于微表面自遮挡,因此实际计算出的结果会比理想结果亮,所以加上 G 项使得结果变暗接近理想结果)，在接近 grazing angle 时遮挡现象最大也就是接近于 0,垂直看向时无遮挡也就是 1。

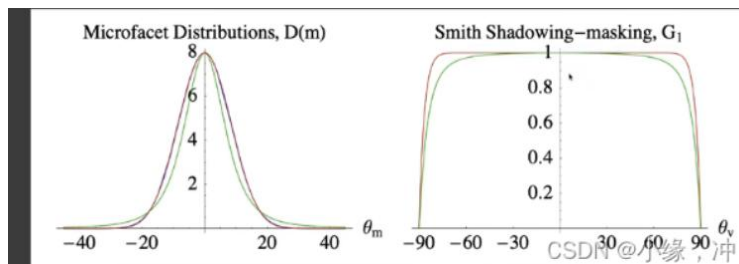


不考虑 G 项时，会导致我们看到的这张图整个外圈是白的。

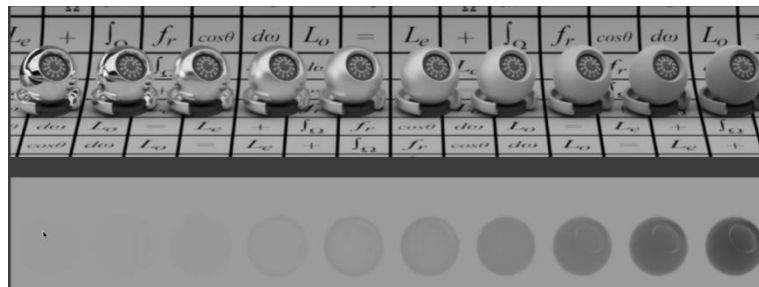
假设我们从 grazing angle 处看向 F0 旁边的白圈里的点,即我们的观察方向与表面法线接近垂直,F 则=1,不考虑 G 项,那 G 项=1,D 是一个法线分布,分母中存在的两个法线与入射方向 (n, i) 法线与出射方向 (n, o) 的点乘，当在 grazing angle 时入射方向、出射方向与法线角度接近 90° ，因此点乘结果会非常小接近 0，导致结果变的巨大。

The Smith Shadowing-Masking: 把 shadowing 和 masking 分开考虑。

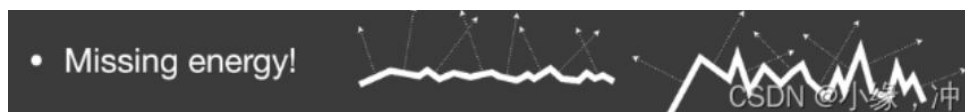
绿线是 GGX,红线是 Beckmann。在两种不同 NDF 下预测出的 G 项有一些细微差别,垂直入射的时候 Shadowing-Masking 不起作用,在 grazing angle 时,G 项变得非常小接近于 0。



问题：能量流失



随着粗糙程度变大,我们渲染得到的结果却越暗,即使认为最左边是抛光,最右边的是哑光。这是因为当表面越粗糙的时候,反射光更容易会被表面挡住,同时越粗糙的表面在表面之间弹射的次数越多,由于没有考虑光线在表面上的多次弹射,只考虑了微表面遮挡的情况,当粗糙度越来越大的时候,能量是不守恒的,因此才导致了粗糙度增大引起了能量损失。



The Kulla-Conty Approximation: 通过经验去补全多次反射丢失的能量,其实是创建一个模拟多次反射表面反射的附加 BRDF 波瓣 $\rightarrow f_{ms}$, 利用这个 BRDF 算出消失的能量作为能量补偿项。把 BRDF (这里的 BRDF 指的是考虑了 Shadowing-Masking 的 BRDF)、Lighting、cos 在一起在整个半球上进行了积分。

$$E(\mu_o) = \int_0^{2\pi} \int_0^1 f(\mu_o, \mu_i, \phi) \mu_i d\mu_i d\phi$$

Note: $\mu = \sin \theta$

- 1.我们认为任何方向入射的 Radiance 是 1,也就是 rendering equation 中的 Lighting 项是 1 (因为是 1 所以式子中没有出现)
- 2.同样假设 BRDF 是各向同性的,也就是与 i、o 无关的;
- 3.因此最终积分的结果意义是,在 uniform 的 lighting=1 的情况下,在经历了 1 bounce 之后射出的总能量 $E(u_o)$.射出的总能量 $E(u_o)$ 是在 0-1 之间的
- 4.有多少能量被遮挡就是 $1-E(u_o)$
- 5.由于 BRDF 的可逆性,因此除了考虑 o 方向,还要考虑 i 方向 (也就是要考虑入射丢失的和出射丢失的),同时要乘一个归一化的量 c (可以是常量或函数)

$$c(1 - E(\mu_i))(1 - E(\mu_o))$$

- Therefore,

$$f_{ms}(\mu_o, \mu_i) = \frac{(1 - E(\mu_o))(1 - E(\mu_i))}{\pi(1 - E_{avg})}, E_{avg} = 2 \int_0^1 E(\mu) \mu d\mu$$

- FYI, validation:

$$\begin{aligned} E_{ms}(\mu_o) &= \int_0^{2\pi} \int_0^1 f_{ms}(\mu_o, \mu_i, \phi) \mu_i d\mu_i d\phi \\ &= 2\pi \int_0^1 \frac{(1 - E(\mu_o))(1 - E(\mu_i))}{\pi(1 - E_{avg})} \mu_i d\mu_i \\ &= 2 \frac{1 - E(\mu_o)}{1 - E_{avg}} \int_0^1 (1 - E(\mu)) \mu d\mu \\ &= \frac{1 - E(\mu_o)}{1 - E_{avg}} (1 - E_{avg}) \\ &= 1 - E(\mu_o) \end{aligned}$$

CSDN @小缘, 冲

设计一种可以交换输入输出方向的 $brdf \rightarrow f_{ms}$, 使得它的积分结果正是我们所失去的能量, 因此用失去的+射出的能量达到能量守恒。

E_{avg} 的积分:

$$\bullet \text{ But } E_{avg}(\mu_o) = 2 \int_0^1 E(\mu_i) \mu_i d\mu_i \text{ is still unknown (as analytic)}$$

CSDN @小缘, 冲

其中 $E(\mu)$ 已经是一个二重积分了, 计算复杂, 通过预计算或打表格的方式来解决。 E_{avg} 经过观察积分式可知不管计算多复杂, E_{avg} 只依赖于 μ_o 与粗糙度, 于是我们根据 u_o 和粗糙度打出一张表格:

单次反射的 BRDF 是有颜色时, 先考虑没有颜色的情况, 然后再去考虑他的颜色是什么。

F_{avg} : 平均菲涅尔项, 不管入射角多大, 平均每次反射会有多少能量反射。平均菲涅尔项的计算就是计算在所有角度下, 菲涅尔项的平均值。使用他来表示有多少能量被吸收以表示颜色的吸收作用。

之前的 E_{uo} 是固定方向上看, 整个出去的能量是多少, 也就是说这些能量是不会参与到后续多次的反射中的, 因此我在后续需要的能参与到后续 bounce 的是 $(1 - E_{uo})$

能够直接看到的能量: $F_{avg} * E_{uo}$

一次弹射后能看到的能量 $F_{avg} * (1 - E_{uo}) * F_{avg} E_{avg}$

.....

k 次反射后能看到的能量 $F_{avg}^k * (1 - E_{uo})^{k-1} * F_{avg} E_{avg}$

那么把 0 次到 k 次弹射的结果相加后, 最终得到了下式中的颜色:

$$\frac{F_{avg} E_{avg}}{1 - F_{avg} (1 - E_{avg})}$$

最近几年有人不考虑能量损失直接加 Diffuse 项, 也就是用一个 diffuse+一个 Microfacet, 因为已经采用了微表面模型, 就不能在与宏观表面模型 Diffuse 的假设一同采用, 同样在物理上也是错误的, 能量不能保证守恒。

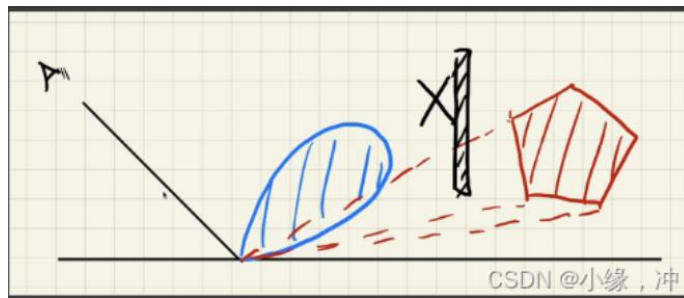
b. LTC-Linearly Transformed Cosines

在多边形光源的照射下快速求出在 ggx 模型上不考虑 shadow 的任意一点的 Shading。

1. 主要是针对 GGX 模型, 对于其他模型原理也同样适用

2. 做的是不考虑 shadow 的 shading

3.光源是多边形光源,且发出的 radiance 是 uniform 的

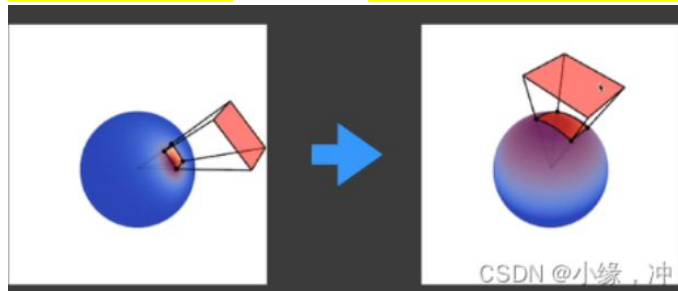


Lobe: Lobe 就是在固定一边方向下的 brdf 的函数图象, 由于 BRDF 是一个 4 维的函数, 在固定一边之后就变为了 2 维的函数。其函数图象像个花瓣。在光源方向 L 固定的情况下, 移动反射方向 R, 当 L 和 R 的半角向量=向量 N 时反射亮度最强。如果继续向上或向下改变 R, 反射亮度都会减弱。

没有 LTC 方法我们需要在多边形光源上我们需要取很多采样点, 并且与 shading point 连线。

1. 在固定入射方向后, 我们将出射的 lobe (brdf 的 lobe) 转变为一个余弦函数, 球面的所有方向经过这个线性变换后使得 brdf 的 lobe 朝向向上。固定了入射方向, 知道了反射方向的 lobe, 通过一个线性变换, 我们将 lobe 内的所有朝向转变为右图中, 其朝向向上的范围, 并且它的覆盖是从正中间是最大值, 依次向周围衰减, 也就是像余弦函数。

2. 在转换 Brdf 的 lobe 时, 将多边形光源也进行变换, 将四边形光源的四个顶点与 shading point 相连得到四个方向(向量), 让这四个向量也进行相同的线性变换, 从而形成新的四边形光源。



我们将 shading point 点任意 brdf 的 lobe 在任意多边形光源下积分求 shading 的问题转变为在一个固定 cos 函数下对任意的多边形光源积分求 shading

- Observations
 - $BRDF \xrightarrow{M^{-1}} \text{Cosine}$
 - Direction: $\omega_i \xrightarrow{M^{-1}} \omega'_i$
 - Domain to integrate: $P \xrightarrow{M^{-1}} P'$

任意一个 cos lobe 可以变换为 brdf 的 lobe, 反之也成立。

1. 任意 brdf 的 lobe 变为固定的 cos lobe

2. 所有的方向 ω_i 进行变换, 变为新的 ω'_i

3.原本的多边形光源所覆盖的方向也需要进行变换,从而产生新的多边形光源覆盖的区域,然后我们在新的区域内对 $\cos$ 进行积分求出 shading 值

• Approach

- A simple change of variable $\omega_i = \frac{M\omega'_i}{\|M\omega'_i\|}$

$$L(\omega_o) = L_i \cdot \int_P F(\omega_i) d\omega_i$$

$$= L_i \cdot \int_P \cos(\omega'_i) d\frac{M\omega'_i}{\|M\omega'_i\|}$$

$$= L_i \cdot \int_{P'} \cos(\omega'_i) J d\omega'_i \quad \text{— Analytic!}$$

CSDN @小缘, 冲

我们认为多边形光源内的任意 radiance 都是 uniform 的,因此 lighting 项可以拆出来,把剩下的 brdf 和 $\cos$ 合在一起 $F(\omega_i)$ 。然后把 $F(\omega_i)$ 通过某种线性变换把所有的 ω_i 变为新的方向 ω'_i ,从而使 $F(\omega_i)$ 变为 $\cos(\omega'_i)$ 。

是一个球面分布函数(余弦分布函数)我们是让一个球面分布函数通过线性变化变成了另一个球面分布函数。

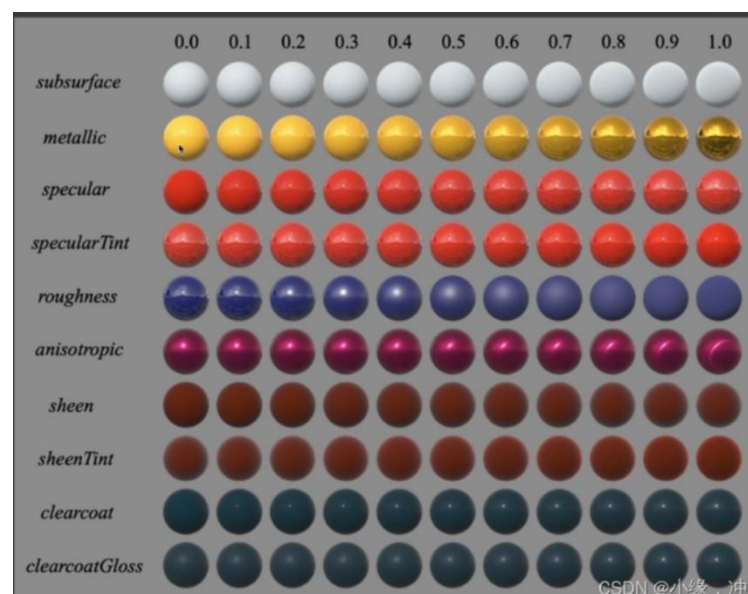
LTC 就是把变化 brdf 和变化光源通过线性变换变成了在固定的 brdf 下变化的光源问题的一个方法。

c. Disney's principle BRDF

微表面模型的效果虽然很好,但是不能表示出所有的材质(多层材质),并且微表面模型对艺术家来说并不好用。Disney's principle BRDF 诞生的首要目的就是为了让 artist 使用方便,因此它并不要求在物理上完全正确。

设计原则:

- 1.应该使用更直观的名词而不是使用物理名词参数,比如使用平缓,饱和度等
- 2.让 brdf 框架不太复杂,也就是让参数数量少一点
- 3.最好有一个拖动条左边最小值,右边最大值供艺术家们进行调整
- 4.有时候为了特殊的效果允许将参数值超过范围,也就是允许小于 0 或大于 1
- 5.所有参数的组合应尽可能可靠和合理,也就是不论如何调整参数最后的结果应该是正常的。



优点:

- 1.容易理解和使用各参数(属性)
- 2.参数的混合组合使得可以在一个模型上显示出很多不同的材质.
- 3.开源

缺点:

- 1.并不是完全基于物理的
- 2.巨大的参数空间使得拥有强大的表示能力,但是会造成冗余现象

d. Non-Photorealistic Rendering(NPR) (非真实渲染)

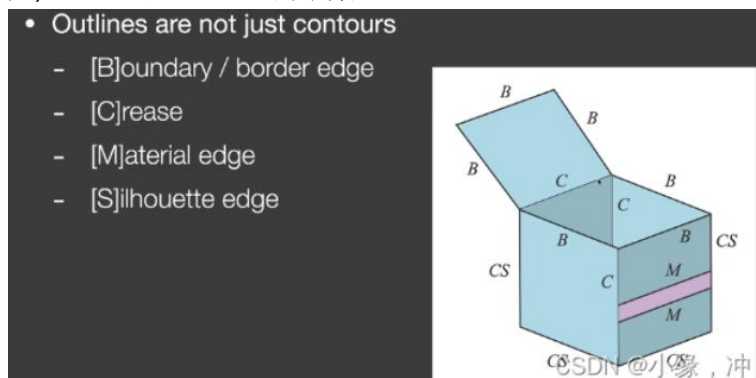
游戏的画风是各不相同的, NPR 的任务就是为了把渲染出的结果偏向于这种卡通的风格。NPR 通常是一些轻量级的处理,一般是在 shader 中做一些简单但是很聪明的处理从而完成风格化。

Photorealistic Rendering 强调画面的真实感,无法分辨是照片还是渲染出来的,其光照,阴影,材质等效果需要无限接近于真实的。

为了制造一种 artistic 的效果,从而使得渲染结果虽然远离真实感。先得到真实渲染的结果,然后再进行变换。NPR 在游戏领域运用的其实是很广泛的,还应用在艺术、可视化、示意图、教育、娱乐等。NPR 保留了很多 photorealistic 的东西。

outline rendering(轮廓渲染)

平常我们说的边缘指的是 contours,整个人物轮廓外围的一圈。Outline 的概念范围很大,contours 是 outline 的子集。



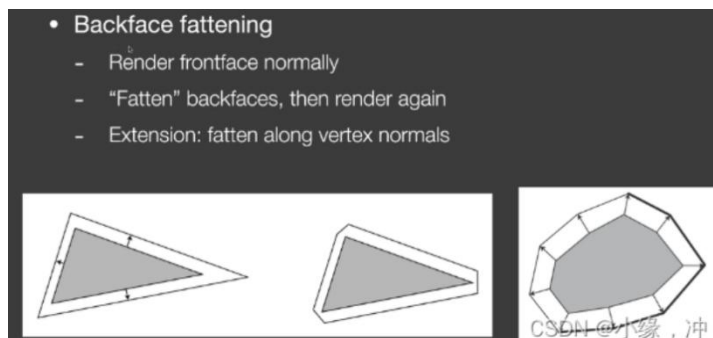
- 1.Boundary/border edge->物体外边界上
- 2.Crease->折痕,通常是在两个表面之间的
- 3.Material->材质边界
- 4.Silhouette edge->必须是在物体外边界上且由多个面共享的.

描边: shading、后期处理

基于法线着色:在 Pixel Shader 中对观察方向向量和着色表面法线向量点乘,如果得到的值接近于 0,说明着色区域是 contour edges.做的是 Silhouette edge



基于几何着色:当我们去渲染一个物体/模型时候,如果我们把模型加大一圈,然后大模型放在原模型背后并且把新模型完全渲染成黑。

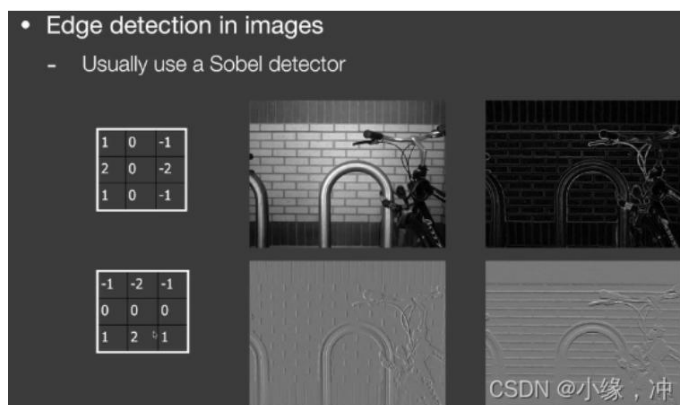


一个封闭模型分为看得见的面为正面,看不见的面为背面, 封闭模型正面和背面所能看到的区域是完全一样的,也就是看不到模型背面的任何信息,因此我们将模型背面给扩大一圈,也就是把背面的每个三角形扩大了一圈,并渲染成黑色,从而得到描边的效果。

主流的方法是将我们将背面的每个顶点沿着顶点的法线方向向外偏移,从而达到背面外拓。

基于图像的后期处理:

先生成未描边的正常渲染结果,再通过一些图像处理的方法从而得到这些边并且标上颜色。



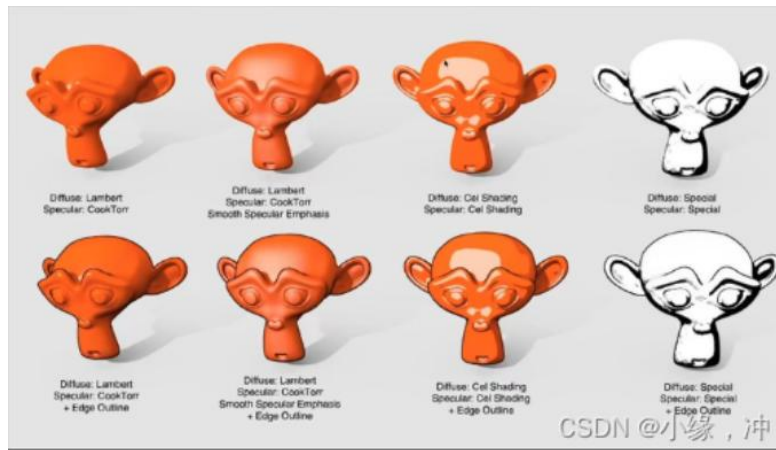
使用图像的 **filter**(卷积), 在一个像素,取周围一圈的值做一个加权平均, 如第一行, **kernel** 竖着几乎没有变换而横着变化非常强烈,当一个像素周围是很平滑的,可以认为值几乎不变,因此在经过这个 **kernel** 后左右两边的值可以认为被抵消掉了。

Color blocks



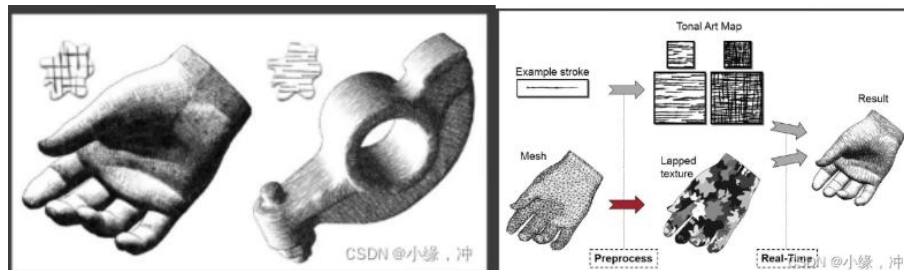
首先得到正常的 **Shading model** 算出来的结果,之后我们进行一个阈值化的操作(**thresholding**), 阈值化操作可以设定多个值, 而拥有更多的色块。

可以根据自己需要去将不同风格的结果组合在一起,比如 **diffuse** 和 **specular** 都进行阈值化处理,或者只处理 **spcular** 不处理 **diffuse** 等组合。



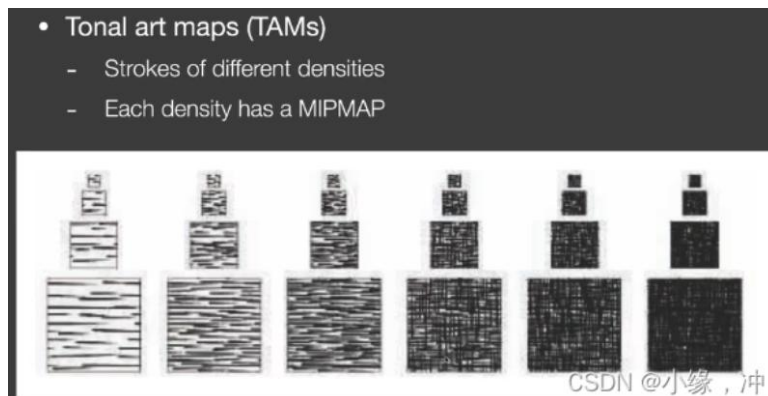
Strokes Surface Stylization

有时候我们并不想得到色块的感觉,而是想得到素描一样的效果,我们知道在素描上,往往越暗的地方需要更多的涂抹,我们把它理解成小格子,越暗的地方格子越多也就是密度越大,因此我们把格子密度与明暗度联系在一起。



TAMs 方法是一种实现类似手绘素描风格渲染的方法:

- 1.关于不同明暗设计不同的纹理
- 2.对于不同明暗的纹理设计其密度不变的 mipmap



四、Real Time Ray Tracing(RTRT)

a. RTX: RTX 其本质上只是硬件的提升,并不涉及任何算法部分,在显卡加装了一个用来做光追的部件,由于光线追踪做的是光线与场景的求交,结合 games101 我们知道光线要经历 BVH 或者 KD-tree 这种加速结构,也就是做一个树的遍历从而快速判断光线是否与三角形求交,这部分对于 GPU 来说是不好做的,因此 NVIDIA 设计了专门的硬件来帮助我们每秒可以 trace 更多的光线。

10 Giga rays per second == 1 sample per pixel

1 sample per pixel 就是每个像素采样一个样本.从而得到最后的光追结果。

b. 减噪要求: 目标为 1SSP 下:

1.质量足够好 (no overblur, no artifacts)

2.速度足够快(小于 2ms 内完成降噪操作)

很多方法在这不适用:

1.Sheared filtering serie(SF,AAF,FSF,MAAF,...)Sheared 方法不行

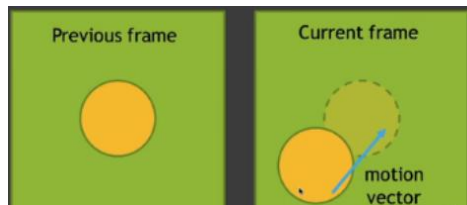
2.Other offline filtering methods (IPP,BM3D,APR,...)离线方法不行

3.Deep learning series(CNN,Autoencoder,...)深度学习不行

c. Industrial Solution-temporal

首先假设整个看的场景的运动是连续的,就是 camera 以某种轨迹看向不同的物体,帧与帧之间有大量的连续性。

motion vector:物体在帧与帧之间是如何运动的,知道某个点在上一帧里点对应的位置。



本质上是时间上有记忆的一种复用,找某一点在不同帧上的对应关系来得到结果。

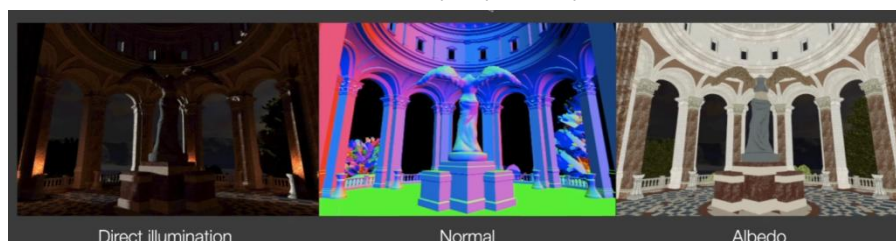
1.认为当前帧是需要去进行 filtering,前一帧时已经 filtering 好的。

2.利用 motion vector 来知道当前帧某一点在上一帧的对应位置。

3.由于我们认为场景运动是连续的,所以认为 shading 也是连续的,上一帧得到降噪好的结果比如颜色之类的,可以在当前帧复用,相当于增加了 spp,但并不是简单的上一帧 1 spp+当前帧 1 spp,由于是一个递归,因此上一帧也复用了上上一帧的结果,因此 spp 其实是很多的,可以理解为一个指数衰减,每一帧都有百分比贡献到下一帧。

d. G-Buffers-几何缓冲区:

在 screen space 我们可以得到很多的信息,可以从 camera 看去通过 screen space 生成深度图,法线图, Albedo 图等,也可以存储 per pixel depth,normal,世界坐标。



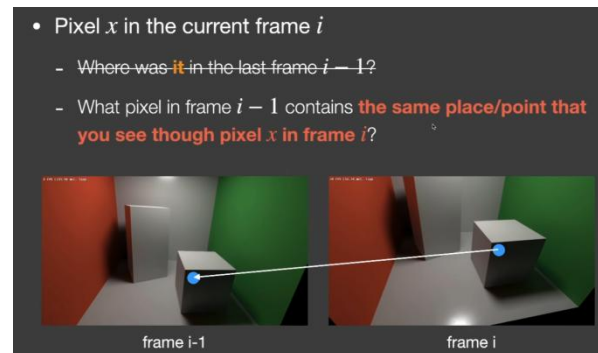
e. Back projection

通过 Back projection 方法求出了 motion vector

frame i: 当前帧

frame i-1: 上一帧

我们找的不是像素的位置，而是当前帧像素里包含的内容在上一帧哪一个像素里。当前帧 (frame i) 中蓝点这个像素我们所得到的点的世界坐标, 投影到上一帧 (frame i-1) 中对应的是哪个像素。



1. 一个点的世界坐标需要通过 MVP 矩阵和视口矩阵从而得到在 screen space 上的坐标，E-viewport 变换，从屏幕空间到世界坐标则依次乘逆矩阵。
2. 若上一帧到这一帧发生了变换，则采用逆变换。将当前帧世界坐标乘以帧移动的逆矩阵就得到了上一帧中这个点的世界坐标。
3. s' 为上一帧世界坐标已知，MVP 和视口变换已知，可以求出上一帧屏幕坐标。

Back projection

- If world coord s is available as a G-buffer, just take it
- Otherwise, $s = M^{-1}V^{-1}P^{-1}E^{-1}x$ (still require z value)
- Motion is known: $s' \xrightarrow{\hat{T}} s$, thus $s' = T^{-1}s$

f. Temporal Accum./Denoising

得到了 motion vector 之后, 我们就可以把当前帧 (noisy 的图) 和上一帧 (没有 noisy 的图) 线性结合在一起。

- Let's denote:
 - $\sim$: unfiltered
 - $-$: filtered
- This frame (i-th frame)

$$\bar{C}^{(i)} = Filter[\tilde{C}^{(i)}]$$

$$\bar{C}^{(i)} = \alpha \bar{C}^{(i)} + (1 - \alpha) C^{(i-1)}$$

$\sim$: unfiltered 表示没有 filter 噪声挺多的内容

$-$: filtered 没有噪声或者噪声比较小的

首先对当前帧进行自己的降噪, 让他不那么 noise, 然后在时间上, 使用当前帧的结果和当前帧对应的上一帧 (已经降噪了的) 做线性 blending. 其中 α 表示平衡系数, 当前帧的贡献, 在 (0.1-0.2) 之间。

时间上的降噪的过程:

- 1.如果在进行 rasterization(primary ray)时得到了世界坐标信息的图存储在 G-buffer 中
- 2.如果没有其信息,则在当前帧的像素通过逆视口变换和逆 MVP 变换得到世界坐标
- 3.已知 motion 矩阵,将世界坐标逆 motion 得到上一帧的世界坐标
- 4.将上一帧的世界坐标进行正 MVP 变换和视口变换得到当前帧中的像素在上一帧中的屏幕位置
- 5.将两个值线性 blending 起来从而得到当前帧最后的结果

缺点:

- 1.switching scenes (burn-in period) -突然切换场景,由于我的上一帧并没有新场景的任何信息因此得到的结果肯定不准确,或者切换光源。
- 2.walking backwards in a hallway (screen space issue)-在倒退的时候周围的信息会不断的增多,因此我们当前帧新出现的物体无法在上一帧的屏幕空间内找到相对应的点。
- 3.suddenly appearing background(disocclusion)-上一帧被遮挡,当前帧未被遮挡。会产生这种残影/拖尾的现象。



避免残影:

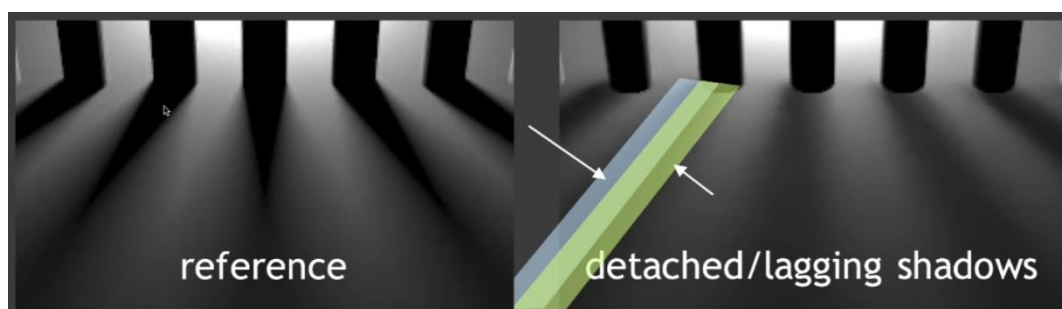
- Clamping $\bar{C}^{(i)} = \alpha \bar{C}^{(i)} + (1 - \alpha) C^{(i-1)}$
 - Clamp previous toward current
- Detection
 - Use e.g. object ID to detect temporal failure
 - Tune α , binary or continuously
 - Possibly strengthen / enlarge spatial filtering
- Problem: **re-introducing noise!**

clamping-把上一帧拉到接近当前帧的结果。

detection-判断是否仍要使用上一帧的结果,在工业界我们渲染是认为每个物体都有自己的编号,如果编号不同,那么认为 motion vector 就不在靠谱了。残影现象没有了,却产生了噪声。

阴影问题:

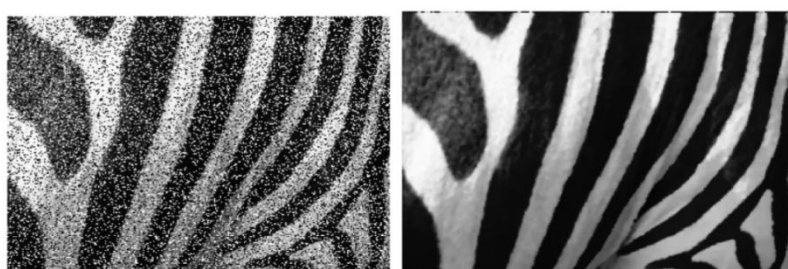
光源从左到右移动,而场景几何并不发生移动, motion vector 是 0,但是由于我们一直在复用未移动的帧结果,就会导致拖影的阴影出现。场景不动的情况下 motion vector 为 0,如果 shading 改变会出现错误的显示。



地板是不动的,因此其上面的每一个像素点的 motion vector 为 0,当我们移动物体时,其地板需要一定的时间适应,之后再在地板上反射出当前场景中的物体,也就是反射滞后。

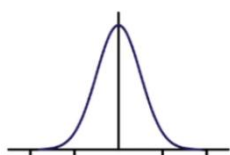
g. Implementation of filtering

滤波 Filtering: 模糊操作, 从一个有噪声的图得到干净的图。采用的低通滤波器只保留低频的信息, 除掉高频的信息。



1. 高频信息中就全是噪声吗? 那么高频中不是噪声的信息被除掉会不会导致信息丢失?
2. 低频信息中就没有噪声吗?

为了给光线追踪由于蒙特卡洛产生的噪声降噪时, 用的是高斯的滤波器, 它的滤波核为类似于正态分布, 中心值高, 向两边衰减



```
For each pixel i
    sum_of_weights = sum_of_weighted_values = 0.0
    For each pixel j around i
        Calculate the weight w_ij = G(|i - j|, sigma)
        sum_of_weighted_values += w_ij * C^{input}[j]
        sum_of_weights += w_ij
    C^{output}[I] = sum_of_weighted_values / sum_of_weights
```

对于任何一个中心像素 i

我们需要定义权值和(sum_of_weights), 加权贡献值的和(sum_of_weighted_values)

对于中心像素 i 周围一圈的任意像素 j (包括像素 i 本身)

根据像素 i 和像素 j 之间的距离和高斯的 σ 找对应的 j 贡献给 i 的值(权值) w_{ij}

将权值 w_{ij} 与像素 j 对应的颜色值相乘得到 j 的加权贡献值累加

将权值 w_{ij} 累加到 sum_of_weights

进行归一化 $\text{sum_of_weighted_values} / \text{sum_of_weights}$ 从而得到像素 i 最终的结果

1. 归一化: 分子是一个加权的 visibility 求和, 分母则是权的求和也就是 $\text{sum_of_weighted_values} / \text{sum_of_weights}$.
2. 高斯的滤波核下, sum_of_weights 不会为 0
3. 像素 j 输入的值 i 可能是一个多通道的值

h. Bilateral filtering-双边滤波

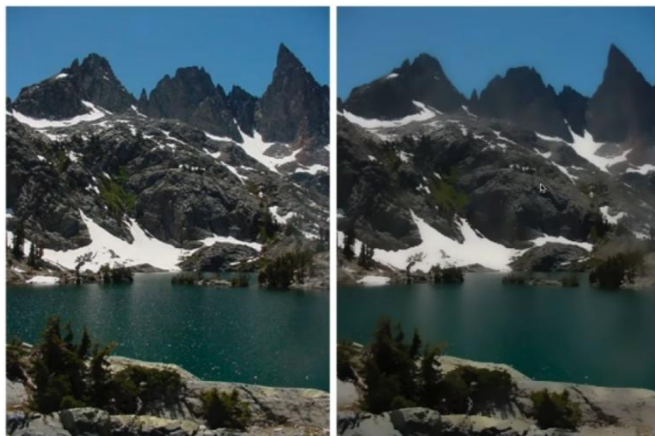
通过高斯我们得到了一个整体都被模糊的结果,但是 we 想让边界仍然锐利, 双边滤波 (Bilateral filtering) 将颜色变化特别剧烈的地方认为是边界。



保留边界的信息（高频）：我们认为颜色变化特别剧烈的地方认为是边界,如果二者差距不是特别大我们继续用高斯处理 j 到 i 的贡献。如果像素 i 和像素 j 之间的颜色值差距过大,我们认为这两个像素分别在边界的两边,从而让像素 j 给 i 的贡献变少,就不会出现边界的模糊情况。

$$w(i, j, k, l) = \exp \left(-\frac{(i - k)^2 + (j - l)^2}{2\sigma_d^2} - \frac{\|I(i, j) - I(k, l)\|^2}{2\sigma_r^2} \right)$$

i 和 j 代表一个像素,k 和 l 代表另一个像素,前半部分为高斯滤波,如果 ij 之间的差距过大,则让 j 给 i 的贡献变小, $I(i, j)$ 表示第一个像素的值, $I(k, l)$ 表示第二个像素的值,他们之间的差异就是分子部分,如果差异过大,就相当于在原本的高斯核上乘上了一个指数（负距离的平方）距离差异越大,整体接近 0。



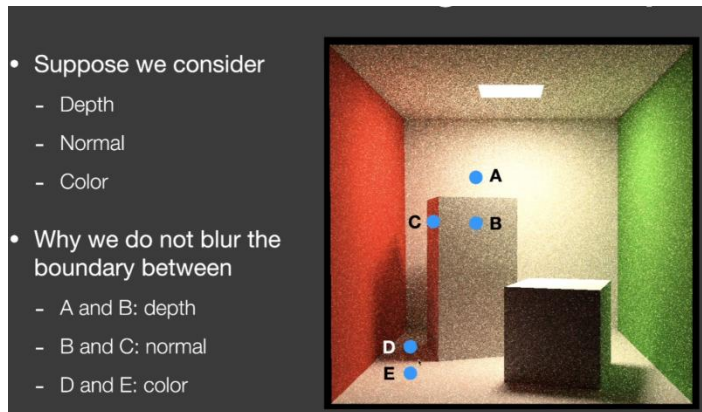
通过双边滤波我们对左边的原图进行处理,可以发现山上的细节和湖面的细节被很好的模糊了,但是边界仍完美的保留了下来,这就是我们想要得到的结果,保留了低频信息但也保留了边界,但有可能会分不清噪声和边界。

i. Joint Bilateral filtering-联合双边滤波

Gaussian filtering 判断两个像素之间的绝对距离, Bilateral filtering 两个像素之间的距离, 和颜色之间的距离。G-BUFFER 完全没有噪声, 与 G-buffer 有关。

在 RTT 中得到的结果有很明显的噪声: 如果用高斯,我们是通过两个像素之间的绝对距离找对应贡献, 双边滤波的话在考虑距离的同时还会考虑两个之间的颜色变化, 因为 A 和 B 都

很 noise，双边滤波得到的结果是不准确的。



1.如果用高斯,我们是通过两个像素之间的绝对距离找对应贡献,双边滤波的话在考虑距离的同时还会考虑两个之间的颜色变化,因为 A 和 B 都很 noise, 双边滤波得到的结果是不准确的。

2.引入深度: 从 G-buffer 得到各自的深度信息,我们可以看到 B 的深度比较浅,而 A 的深度比较深,所以我们不希望 A 有太多的贡献到 B 上。

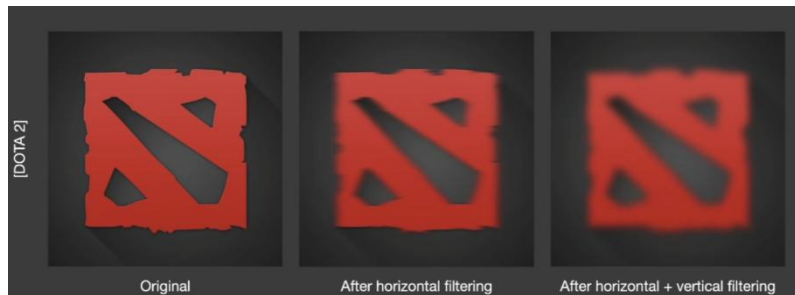
3.引入法线: B,C 二者之间的深度是差不多的,BC 法线朝向是不一样的。联合双边滤波的本质就是在 kernel 里多算几组不同 feature 下的贡献,并将其相乘得到最后的结果。

j. Implementing Large filters

滤波: 对于任何一个像素,我们需要考虑他周围 $N * N$ 个像素的贡献值,加权平均之后写回这个像素上。如果是 large filter,也就是如果这个 $N * N$ 很大,就会使得 filter 变得特别慢。

方法一: Separate Passes(拆分实现)

有一张图,我们先将它在水平方向上 filter 一遍,之后再在竖直方向上 filter 一遍,也就是将本来一趟 $N * N$ 的,分成两趟来做,水平的 $1 * N$ 和竖直的 $N * 1$ 。



如果我们使用 $N * N$ 的 filter,对于任何一个像素我们需要访问 N^2 个纹理。但如果我们将其拆分为两趟,第一趟访问 N 个纹理,第二趟访问 N 个纹理,总共访问了 $2N$ 个纹理。

2D 的高斯 filter 拆分为两个 1D 的高斯 filter: 高斯函数本身就是可拆分的

A 2D Gaussian filter kernel is separable

$$G_{2D}(x, y) = G_{1D}(x) \cdot G_{1D}(y)$$

Recall: filtering == convolution

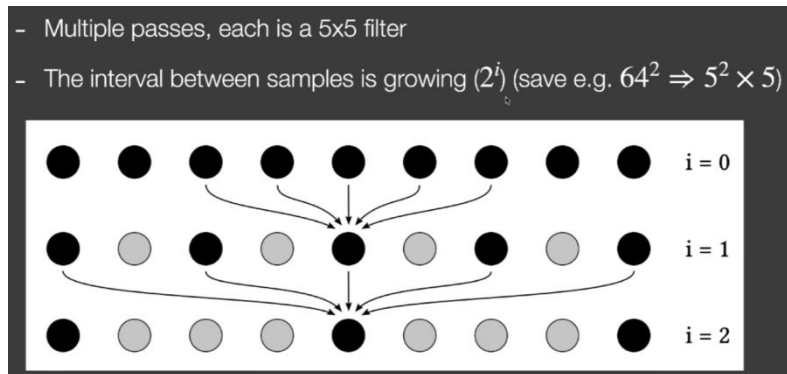
$$\iint F(x_0, y_0) G_{2D}(x_0 - x, y_0 - y) dx dy = \int \left(\int F(x_0, y_0) G_{1D}(x_0 - x) dx \right) G_{1D}(y_0 - y) dy$$

filtering == convolution 滤波 == 卷积

理论上, 由于联合双边滤波 filter 函数复杂, 不能用拆分滤波, 但是还是在用。

方法二：Progressively Growing Sizes（渐增大滤波范围）

我们不希望对于任何一个像素进行 $N * N$ 次的纹理查询,采用一个逐步增大的 filter,比如先用一个小的 filter,然后用中号的 filter,最后是大号的 filter,通过多趟的 filter 得到 $N*N$ 的 filter 得到的结果。



多趟操作,每趟都是 $5 * 5$ 大小的 filter, 第一趟是 $i=0$,第二趟 $i=1$不同趟数的 $5*5$ 的 filter 中间是有间隔的,在第 i 趟时,样本之间的间隔是 2^i ,但每次都要采样 5 个。

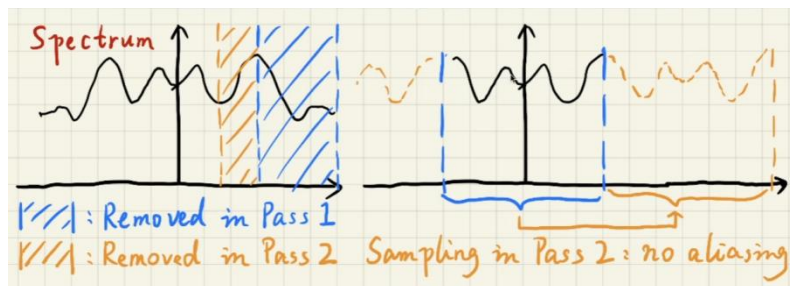
假设要做 5 趟,每趟都是 $5 * 5$ 的 filter,那么 i 在第五层时候,样本之间的间隔是 $2^4 = 16$,第五层一共要五个样本,也就是 4 个间隔,大小为 64。也就是说在第五层时候我们相当于做了一个 $64 * 64$ 的 filter,但我们对于这个像素来说,我们做了五趟,每趟 $5 * 5$ 大小的 filter,一共做了 125 次的纹理查询,对于 $64 * 64 = 4096$ 这个数字来说是很小。

1.为什么要逐渐增大 filter,而不是一上来就使用第五趟中间间隔 16 个样本的 filter?

用更大的 filter == 除掉低频信息,在第一趟时候,通过一个小范围的 filter,将频谱上的高频信息（蓝色区域）除掉了,依次类推直到最后一趟除掉最低频上的信息。

2.为什么可以采样间隔那么多的样本?

采样 = 重复的搬移频谱,第一趟 pass 中出去了高频蓝色区域,然后在第二趟 pass 中相当于在 $9 * 9$ 的 filter 中采样出 $5 * 5$ 的 filter,我们采样的间隔对应到频谱上正好是右半图的蓝色部分,也就是在第一趟除掉高频信息后的区域的 2 倍。



k. Outlier Removal

用蒙特卡洛方法渲染一张图时,得到的结果会出现一些点过亮或者过暗,如果有一个点过亮,在经过这个 $7*7$ 的 filter 处理过后,会影响一块区域,使得它这一块区域变亮。



Outlier Detection and Clamping

Detection: 对于每个像素,我们都取他们周围一个小范围的区域,例如 7*7 或者 5*5,然后把这个范围内颜色的均值(mean)和方差(variance)给算出来,我们认为正常的范围在均值+若干方差内,超过这个范围的我们认为他是 outlier。

Clamping: 如果在范围内我们找到了 outlier 的点,我们把这个点的值给 clamp 到接近范围的值。

Temporal Clamping: 当 motion vector 为 0 导致的残影现象,当前帧与上一帧中对应的信息差异过大时,我们把上一帧中的信息值给 clamp 到接近当前帧的信息值。如果上一帧对应的值超出这个范围,则把他 clamp 到范围内,再和当前帧做线性 blending,从而得到当前帧 noisy-free 的结果。

I. Specific Filtering Approaches for RTRT

SVGF-Basic Idea: ground truth 和 svgf 结果非常接近,SVGF 的方法与在时空上降噪的方法差不多,加了一些 variance analysis 和 tricks.

SVGF-Joint Bilateral Filtering:

Depth:

- **Depth**
$$w_z = \exp\left(-\frac{|z(p) - z(q)|}{\sigma_z |\nabla z(p) \cdot (p - q)| + \epsilon}\right)$$
- Understanding:
 - A and B are on the same plane, of similar color, so they should contribute to each other
 - But the depth between A and B are very different!
 - Therefore, it is preferred to use the depth difference **w.r.t. the tangent plane**

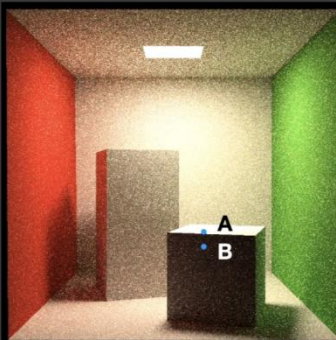


A,B 侧向我们,所以深度差距比较大,按照深度来决定贡献不太合理,可以看沿着法线的那个面投影出来的深度差异。 $\exp(x)$ 是返回 e 的 x 次方。

深度的梯度就是往某一方向上的变化率,因此当我们知道 A 和 B 之间的距离,之后用 深度梯度 X 距离 = 实际深度变化量(垂直于平面法线上的变化 * 距离 = 实际深度变化量)

Normal:

- 3 factors to guide filtering
 - **Normal**
$$w_n = \max(0, n(p) \cdot n(q))^{\sigma_n}$$
 - Recall, does not have to be a Gaussian
 - Note: in case normal maps exist, use macro normals



我们用两个点法线向量求一个点积,由于求出来的值有可能是负值,因此使用 $\max()$ 把负的值给 clamp 到 0。如果场景中运用了法线贴图来制造凹凸效果,我们再判断时运用平面原本的法线,而不是为了制造凹凸效果而改变过的法线。

Luminance:

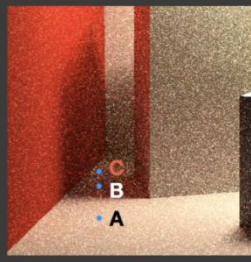
- 3 factors to guide filtering

- **Luminance** (grayscale color value)

$$w_l = \exp\left(-\frac{|l_i(p) - l_i(q)|}{\sigma_l \sqrt{g_{3 \times 3}(\text{Var}(l_i(p)))} + \epsilon}\right)$$

- Variance

- Calculate spatially in 7x7
- Also averaged over time using motion vectors
- Take another 3x3 spatial filter before use



在考虑颜色差异时，最简单就是应用双边滤波里给的颜色差异来考虑，比如我们将 RGB 转换为 grayscale（灰度），这种颜色我们称其为 luminance。由于噪声的存在会出现一些干扰，也就是 B 点虽然在阴影里，但是可能刚好选择的点是一个噪声，也就是其特别亮，此时 A 特别亮，B 也特别亮，那么 A 和 B 就会互相贡献。

SVGF 中 V-variance 方差：我们在考虑 A 点是否贡献到 B 点时，先看 A 和 B 点之间的颜色差异，并且除以 B 点周围的一个标准差。在标准差大的时候更不愿意去相信颜色差异。

spatial filter --> temporal filter --> spatial filter

B 点的 variance：在点的周围取一个 7*7 的区域算出区域里的 variance，同样的操作在时间上累积下来（平均），找上一帧中对应点的 variance，从而通过 motion vector 在 temporal 上把 variance 信息给 filter 下来，这样先 spatial filter 求出 variance 之后，再随时间平均从而得到了一个比较平滑的 variance 值。最后在使用 variance 时如果不放心，我们再在周围取一个 3*3 的区域做一次 spatial filter 得到 variance。

SVGF 会产生“残影”现象：当一个场景固定，我们只移动光源时候，阴影会随着光源的移动而变化，当前帧会复用上一帧的阴影。

m. RAE（Recurrent AutoEncoder）

一种结构，对 Monte carlo 路径追踪得到的结果进行 reconstruction-对 RTRT 做滤波。

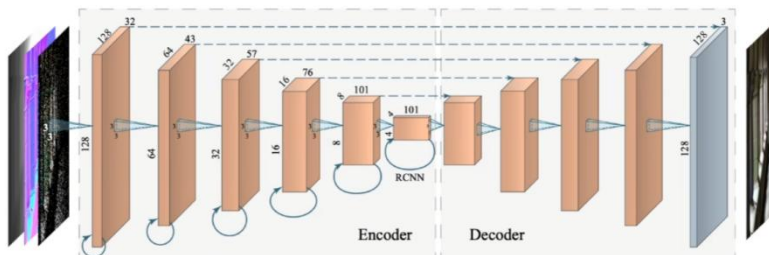
后期处理，把 noise 的图变 clean。

使用 G-buffers

神经网络会自动将 temporal 的结果累积起来

AutoEncoder,是一个漏斗形的结构，输入经过若干层神经网络后变成一个很小的东西，之后再再小的东西不断地展开，类 U 型神经网络。

AutoEncoder 每一层都有连向他自己的。可以利用历史的信息，每一层神经网络有一个 recurrent 连接，也就是每一层神经网络不仅可以连向下一层，也可以连回自己这一层。-从之前的帧中积累(并逐渐忘记)信息



五、Glimpse of Industry Solution（工业界）

a. Temporal Anti-Aliasing(TAA) 在时间上的抗锯齿或反走样
why aliasing?

Not enough samples per pixel during rasterization 一个像素中的样本数量不足

Therefore, the ultimate solution is to use more samples 终极解决方案就是用更多的样本，也就是 101 中的 MSAA

b.